基于红外干涉法测量碳化硅外延层厚度的研究

摘要

碳化硅（SiC）外延层厚度是决定其电力电子器件击穿电压与导通电阻的关键参数。傅里叶变换红外光谱法（FTIR），作为一种基于光学干涉原理的无损检测技术，是测量该参数的核心手段。本文针对碳化硅及硅外延层的厚度测量问题，构建了相应的数学模型，并提出了一套通用的厚度计算方法。

针对**问题**一，本文分析了简化的双光束干涉模型，该模型考虑了外延层上表面（空气-外延层）与下表面（外延层-衬底）反射光的相干叠加。基于**斯涅尔定律**与**光的相干叠加原理**，重点分析了光程差与半波损失对干涉光谱的影响。通过该模型，我们将外延层厚度的求解问题转化为求解其折射率的问题。进一步，通过引入**介电响应函数**，建立了反射率与折射率之间的定量关系，最终构建了完整的外延层厚度计算模型。

针对**问题**二，本文应用所建模型对实测数据进行计算。首先，采用**傅里叶级数拟合**将离散光谱数据转化为连续函数，并选取折射率变化平缓的波数范围（1700-3000 cm^{-1}）作为有效计算区间。通过提取该区间内的干涉谱极值点，代入问题一的模型，计算得到两种不同入射角下的外延层厚度分别为 **8.071 μm** 与 **7.996 μm**，二者相对差异为 **0.934%**。该结果验证了模型在不同实验条件下的稳健性，且计算精度满足工业应用要求。

针对**问题**三，本文将研究拓展至更复杂的多光束干涉情景，并将其分解为三个子问题：多光束干涉的必要条件推导、相应的厚度计算模型构建以及误差分析。本文推导了多光束干涉需满足的可观测性、相干性及厚度均匀性等条件。在此基础上，采用**广义菲涅尔公式**对原有模型进行修正，通过联立反射率与衬底透射率数据反演求解外延层的折射率，从而计算其厚度。计算得到在 10° 与 15° 入射角下，外延层厚度分别为 **5.1445 μm** 和 **5.2198 μm**。此外，将此通用模型应用于问题二中可忽略多光束干涉的情况，所得结果与先前计算值高度吻合，从而交叉验证了两个模型的准确性与兼容性。

综上所述，本文所构建的基于光学干涉的数学模型能够精准地实现对碳化硅与硅外延层厚度的无损测量。模型计算得到的碳化硅外延层厚度相对差异不高于 **0.934%**，硅外延层厚度相对差异不高于 **1.45%**。该模型兼具理论严谨性与工程实用价值，可为半导体材料的质量控制提供可靠的技术支持。

关键字：介电响应函数　菲涅尔公式　多光束干涉　艾里公式　外延层厚度测量

一、 问题重述

1.1 问题背景

碳化硅作为第三代半导体的核心材料之一，凭借其宽禁带、高热导率、高击穿电场等优异的物理特性，在电力电子、半导体制造、先进磨料及耐火材料等领域展现出巨大的应用潜力，尤其在新能源汽车和光伏发电等高增长行业中扮演着关键角色。

SiC 电力电子器件的核心功能区是构建于单晶 SiC 衬底之上的外延层。该外延层通过外延生长（Epitaxial Growth）技术制备，其晶体结构与衬底保持一致，厚度通常为微米量级。外延层的厚度是决定器件击穿电压、导通电阻等关键性能指标的核心参数，其均匀性与精确度直接影响器件的整体性能、可靠性与成品率。因此，研究一套快速、精确、可靠的 SiC 外延层厚度无损检测方法，对于优化生产工艺与保障产品质量具有重要的科学意义与工程价值。

傅里叶变换红外光谱法（FTIR）是实现外延层厚度无损测量的主要技术手段之一，其原理基于等倾干涉。由于外延层与衬底的载流子浓度存在显著差异，二者对红外光呈现出不同的折射率。当红外光束入射至样品表面时，分别在外延层的上表面（空气-外延层界面）和下表面（外延层-衬底界面）发生反射。这两束反射光因光程差而产生干涉，形成的干涉信号经傅里叶变换后得到波数-干涉光谱反射率数据。通过分析光谱图中的干涉条纹特征（如峰谷位置），结合材料的折射率模型与光的入射参数，即可建立数学模型以计算出外延层的厚度。

1.2 问题要求

问题 1

本问题旨在基于光学干涉原理，需建立一个用于描述外延层厚度测量的数学模型。研究假设仅考虑入射光在外延层上下界面经一次反射与透射后形成的双光束干涉，建立能够精确表征反射光谱波数、反射率与外延层厚度之间定量关系的数学模型。

问题 2

本问题要求基于问题一所建立的双光束干涉模型，设计并实现一套完整的外延层厚度计算程序，并使用该程序处理附件 1 与附件 2 所提供的 SiC 晶圆反射光谱实验数据，求解出外延层的厚度值。完成厚度求解后，还需要对计算结果的可靠性与精度进行评估。

问题 3

本问题旨在探究更加复杂的多光束干涉效应对厚度测量的影响。首先，要求从光学原理出发，分析多光束干涉现象产生的条件，并从理论上分析其对厚度计算精度的潜在

影响。然后，需基于该理论判断附件 3 与 4 中的硅晶圆数据是否出现多光束干涉，并构建相应的数学模型以求解其厚度。最后，需探讨将此模型应用于 SiC 数据的可能性，若适用，则需对问题二的计算结果进行修正与优化。

二、 问题分析

2.1 问题一分析

对于问题一，其核心是基于一个简化的双光束干涉物理情景。该情景假设干涉主要源于两束相干光：一束为从外延层上表面直接反射的光波，另一束为透射进入外延层、经衬底界面反射后再次透射出的光波。这两束光的光程差（Optical Path Difference,OPD）是外延层厚度 d 的函数，并直接决定了最终的干涉光谱形态。

2.2 问题二分析

对于问题二，其核心任务是应用问题一建立的数学模型，设计并实施一套完整的数据处理与计算流程，以求解 SiC 外延层的实际厚度。具体步骤包括：首先，从干涉光谱数据附件 1 和附件 2 中提取两个关键参数——反射率 R 和干涉条纹的波数间隔 $\Delta\tilde{\nu}$；其次，利用提取的 R 值反演出材料的折射率 n_1；最后，将 n_1 与 $\Delta\tilde{\nu}$ 代入核心计算公式17，最终确定外延层的厚度 d。

为评估所建模型的可靠性，本问题设计了一项验证性分析。注意到附件 1 与附件 2 的数据源自同一块晶圆片，仅在入射角（分别为 10° 和 15°）这一实验参数上有所不同。由于外延层的物理厚度是固有属性，不应随测量角度而改变，因此，一个可靠的模型应能从这两组独立的数据中计算出高度一致的厚度值。通过定量比较两个计算结果的差异，可以有效地评估模型的准确性和其对实验条件变化的稳健性。

2.3 问题三分析

对于问题三，首先需要建立多光束干涉模型，该模型基于艾里公式（Airy formula），通过对所有反射光束的复振幅进行矢量叠加，精确描述薄膜系统的总反射率。

然后需要分析多光束干涉的必要条件，主要包括三个方面：(1) 强度可观测性，即相干叠加后的干涉极大值强度需高于探测阈值；(2) 相干性条件，即光程差需小于光源的相干长度；(3) 厚度均匀性，即外延层厚度波动需小于特定阈值以保证干涉条纹清晰。

最后分析四个附件是否满足多光束干涉的必要条件，若会出现在附件 1 和附件 2 中，则需要使用建立的多光束干涉模型计算外延层厚度，消除多光束干涉的影响。

三、 模型假设

为简化问题，本文做出以下假设：

- 假设光源到外延层之间的介质为空气。
- 外延层折射率大于衬底折射率。
- 衬底足够厚，其下表面的反射可忽略。
- 界面反射系数与界面透射系数满足能量守恒定律 $r^2 + t^2 = 1$
- 仅考虑界面反射/透射的光强度衰减, 忽略光在传播过程中的吸收损失。**解释：** 外延层的厚度大约在 11 微米[1]，其对光的吸收极其微小，可以忽略。
- 介质分层界面平整无缺陷
- 入射光是红外非偏振的。
- SiC 是 6H 型的。

四、 符号说明

符号	说明	单位
n_1	空气折射率	无量纲
n_2	外延层折射率	无量纲
n_3	衬底折射率	无量纲
θ_1	入射角	° 或 rad
θ_2	折射角	° 或 rad
m	干涉级数	无量纲
ΔL	光程差	nm
$\tilde{\nu}$	波数	cm^{-1}
r_{01}	空气-外延层界面反射系数	无量纲
r_{12}	外延层-衬底界面反射系数	无量纲
t_{01}	空气-外延层界面透射系数	无量纲
t_{10}	外延层-衬底界面透射系数	无量纲
I_k	第 k 束反射光强度	$\mathrm{W/m^2}$
ε	最小可观测强度比例	无量纲

部分变量未在此处列出，将于每个小节展开讨论。

五、 问题一的模型的建立和求解

5.1 光程差的计算

考虑红外光束从空气介质入射至外延层表面的物理过程。设入射角为 θ_1，折射角为 θ_2。若分别以 n_1 和 n_2 表示空气和外延层材料的折射率，则根据光的折射理论，入射角与折射角之间的关系遵循斯涅尔定律（Snell's Law），其数学表达式如下：

$$n_1 \sin\theta_1 = n_2 \sin\theta_2 \tag{1}$$

干涉现象源于从外延层上、下表面反射的两束光波的相干叠加。由于两束光所经历的传播路径不同，它们之间将产生一定的光程差（Optical Path Difference, OPD），这是决定干涉效果的核心物理量。其具体几何路径如下图所示，其中光程差用红色线段标出。

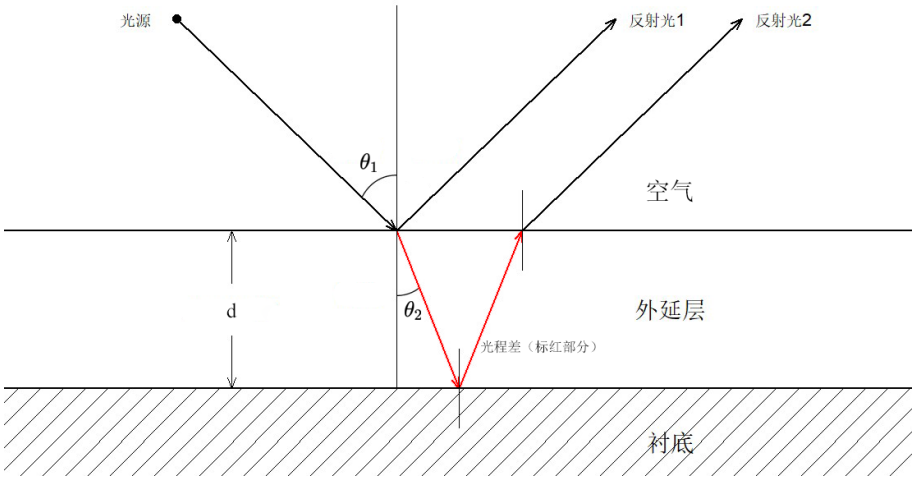

图 1　薄膜干涉光程差推导示意图

设外延层的厚度为 d，折射率为 n_2，光线在层内的折射角为 θ_2。基于图中所示的几何关系，可以推导出由传播路径差异所导致的光程差 OPD 的表达式为：

$$OPD = 2n_2 d \cos\theta_2 \tag{2}$$

此外，完整的干涉模型必须计及光波在界面反射时可能发生的相位突变。根据电磁理论，当光从光疏介质（较低折射率）向光密介质（较高折射率）反射时，其相位会发生 π 的突变，这等效于在光程中引入一个 $\lambda/2$ 的附加量。

在本问题所描述的系统中，需要考虑两次反射：一次发生在空气-外延层界面，另一次发生在外延层-衬底界面。根据假设条件，材料的折射率满足 $n_{空气} < n_{衬底} < n_{外延层}$，那么在空气-外延层界面反射的光波将经历半波损失，而在外延层-衬底界面反射的光波则不会。因此，两束相干光之间存在一个固有的相对相位差 π。将此效应纳入考量后，总的有效光程差应修正为：

$$OPD = 2n_2 d\cos\theta_2 + \frac{\lambda}{2} \tag{3}$$

5.2 干涉极大值的判断

实验中观测到的干涉光谱强度极大值，对应于相长干涉的发生。其物理条件为：两束相干光波的总光程差等于波长 λ 的整数倍。该条件可由下式表述：

$$OPD = m\lambda, \quad m \in \mathbb{Z}^+ \tag{4}$$

其中，m 为干涉级次。

将前述推导出的总光程差表达式（式3）代入该相长干涉条件，可得：

$$2n_2 d\cos\theta_2 + \frac{\lambda}{2} = m\lambda \tag{5}$$

整理上式，可得描述干涉极大值波长位置的核心方程：

$$2n_2 d\cos\theta_2 = \left(m - \frac{1}{2}\right)\lambda \tag{6}$$

在光谱学分析中，通常采用波数 $\tilde{\nu}$ 替代波长 λ 作为自变量，因为波数与能量成正比。二者之间的关系定义为 $\lambda = 1/\tilde{\nu}$。为了使理论模型与实验光谱数据直接对应，我们将此关系代入式6，最终可得到干涉极大值的判断条件：

$$2n_2 d\cos\theta_2 = \frac{m - \frac{1}{2}}{\tilde{\nu}} \tag{7}$$

5.3 外延层折射率的计算

5.3.1 介电响应函数理论框架

针对问题一的建模需求，我们参考了 MacMillan 等学者关于 SiC 外延层厚度测量的研究 [2]。该研究基于介电响应函数理论，建立了通过角频率 ω 计算 SiC 外延层折射率的完整模型。

角频率 ω 与光波频率 f 的关系为：

$$\omega = 2\pi f \tag{8}$$

在光谱测量中，通常使用波数 $\tilde{\nu}$（单位：cm^{-1}）表示光谱位置，其与频率的关系为：

$$\tilde{\nu} = \frac{f}{c} \tag{9}$$

其中 c 为真空中光速（$c \approx 2.998 \times 10^{10}$ cm/s）。

由此可得角频率与波数的转换关系：

$$\omega = 2\pi c\tilde{\nu} \tag{10}$$

SiC 的介电函数 $\varepsilon(\omega)$ 包含两个主要贡献项：

1. **晶格振动贡献**：由横向光学声子 (TO) 和纵向光学声子 (LO) 引起，采用 Lorentz 振荡器模型描述；
2. **自由载流子贡献**：由掺杂产生的载流子引起，采用 Drude 模型描述。

完整的介电函数表达式为：

$$\varepsilon(\omega) = \varepsilon_\infty \left(1 + \mathcal{L}(\omega) - \mathcal{D}(\omega)\right) \tag{11}$$

其中 ε_∞ 为高频介电常数。

5.3.2 Lorentz 振荡器项

对于 6H-SiC 晶体，考虑单一光学声子模式的贡献：

$$\mathcal{L}(\omega) = \frac{\omega_L^2 - \omega_T^2}{\omega_T^2 - \omega^2 - i\Gamma\omega} \tag{12}$$

式中：

- ω_T：横向光学声子频率
- ω_L：纵向光学声子频率
- Γ：声子阻尼系数

5.3.3 Drude 项

自由载流子贡献项表示为：

$$\mathcal{D}(\omega) = \frac{\omega_p^2}{\omega(\omega + i\gamma)} \tag{13}$$

等离子体频率 ω_p 由载流子浓度决定：

$$\omega_p^2 = \frac{4\pi N e^2}{m^*} \tag{14}$$

其中：

- N：自由载流子浓度（cm^{-3}）
- e：电子电荷
- m^*：载流子有效质量
- γ：载流子散射率

5.3.4 材料参数取值

根据文献 [2]，6H-SiC 晶体的关键参数如表1所示。需要注意的是，晶体各向异性导致平行 (‖) 和垂直 (⊥) 于 c 轴方向的参数存在差异。

表 1　6H-SiC 的振子参数 [2]

参数	符号	垂直分量 (⊥)	平行分量 (‖)
高频介电常数	ε_∞	6.52	6.70
横向光学声子频率	ω_{TO}	793.9 cm^{-1}	785.9 cm^{-1}
纵向光学声子频率	ω_{LO}	970.1 cm^{-1}	-
声子阻尼系数	Γ	4.763 cm^{-1}	5.501 cm^{-1}
载流子有效质量	m^*	$0.42m_0$	$1.83m_0$
载流子散射率	γ	757.0 cm^{-1}	757.0 cm^{-1}

5.3.5 折射率计算

最终，外延层的折射率 n 通过求介电函数实部的算术平均值得到：

$$n(\omega) = \sqrt{\mathrm{Re}[\varepsilon(\omega)]} \tag{15}$$

5.4 外延层厚度的计算

通过筛选合适的实验数据，选取折射率 n_2 随波数的变化不显著的数据区间，可以使在两个相邻干涉峰的波数区间内，忽略不计折射率 n_2 和折射角 θ_1 的变化（详见章节六、），由式7有：

$$\Delta\tilde{\nu} = \tilde{\nu}_{m+1} - \tilde{\nu}_m = \frac{m+1-\frac{1}{2}}{2n_2 d\cos\theta_2} - \frac{m-\frac{1}{2}}{2n_2 d\cos\theta_2} = \frac{1}{2n_2 d\cos\theta_2} \tag{16}$$

对上式进行代数重排，即可得到一个直接求解外延层厚度 d 的表达式：

$$d = \frac{1}{2n_2\cos\theta_2\Delta\tilde{\nu}} \tag{17}$$

该方程是问题一模型的核心计算公式。它清晰地表明，只要通过实验光谱数据确定平均干涉条纹的波数差 $\Delta\tilde{\nu}$，并结合由波数 $\tilde{v}$ 反演出的外延层折射率 n_2，即可计算出 SiC 外延层的厚度 d。

六、 问题二的模型的建立和求解

6.1 数据处理

为应用问题一建立的数学模型，首要步骤是对原始光谱数据进行预处理，并选取一个最优的分析区间。根据相关文献 [3]，SiC 材料在 1700–3000 cm^{-1} 的波数范围内表现出较弱的色散特性，即其折射率 n_1 随波数的变化不显著。

这一物理特性为模型应用提供了重要便利：在该区间内，可将折射率 n_1 近似为一个常数进行处理。更重要的是，它允许我们通过对区间内多个干涉条纹的反射率和波数间隔进行统计平均，从而有效抑制局部噪声干扰，提高计算结果的稳健性与精度。因此，本研究确定采用 1700–3000 cm^{-1} 作为数据分析的有效光谱范围。

为了达到良好拟合效果，我们采用了傅里叶级数展开。通过对不同傅里叶级数项数（N）的拟合效果进行系统性评估，并权衡模型的精度与复杂性，最终确定当展开项数设定为 $N = 10$ 时，模型能够实现良好的拟合精度，并有效避免了过拟合现象。

所获得的傅里叶级数拟合公式的一般形式如下：

$$y(x) = a_0 + \sum_{n=1}^{N} \left(a_n \cos\left(\frac{n\pi x}{L}\right) + b_n \sin\left(\frac{n\pi x}{L}\right) \right) \tag{18}$$

其中，$L = 843.46$ 为傅里叶级数展开的半周期，$N = 10$ 为展开的项数。拟合得到的系数列于表2中：

表 2　傅里叶级数拟合系数

谐波序数 n	系数 a_n	系数 b_n
0	17.4664	–
1	-0.4789	-0.5272
2	-0.3188	-0.2425
3	-0.2741	-0.1284
4	-0.2303	-0.0292
5	-0.2247	0.0446
6	-0.2380	0.2427
7	-0.3348	-0.4090
8	-0.0566	-0.1167
9	-0.0187	-0.0172
10	-0.0151	0.0204

为了直观地评估所建立的傅里叶级数模型的拟合效果，图2展示了拟合曲线。此图

清晰地呈现了通过 10 项傅里叶级数展开得到的函数形式与实际数据或预期趋势之间的对应关系，从而验证了选择该展开项数在保证拟合精度的同时。

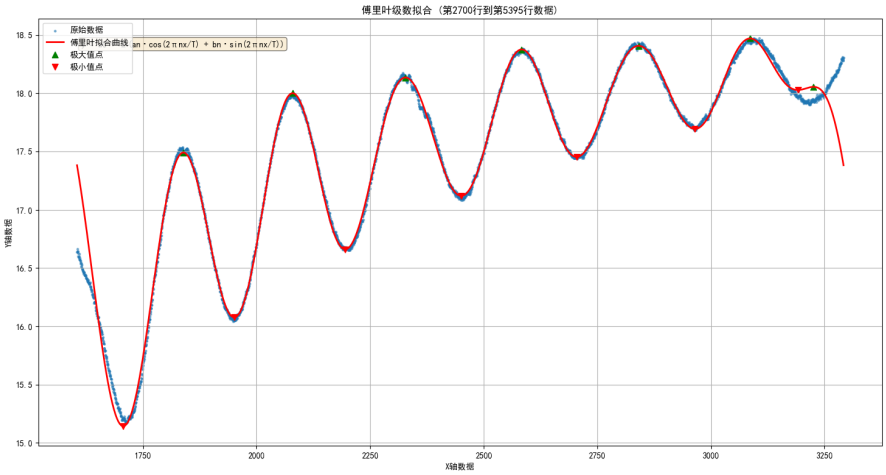

图 2　拟合示意图

我们从附件 1 和附件 2 拟合后的数据中分别提取了该范围内的 6 个干涉极大值与极小值点。下表详细列出了这些特征点对应的反射率值、波数及其波数差（其中 1-6、13-18 为极大值点；7-12、19-24 为极小值点）：

表 3　波数数据表

附件 1			附件 2		
编号	波数	与 n+1 的波数差	编号	波数	与 n+1 的波数差
1	1839.021	240.447	13	1860.271	236.095
2	2079.467	248.319	14	2096.366	268.676
3	2327.786	255.849	15	2365.042	245.069
4	2583.635	257.378	16	2610.111	259.623
5	2841.013	245.311	17	2869.733	252.767
6	3086.324	–	18	3122.500	–
7	1706.716	244.182	19	1724.204	246.614
8	1950.898	244.332	20	1970.818	249.656
9	2195.230	255.500	21	2220.474	257.912
10	2450.730	253.621	22	2478.386	254.228
11	2704.351	260.653	23	2732.614	255.918
12	2965.005	–	24	2988.533	–
平均值		250.5591	平均值		252.6558

注：表中“–”表示该编号对应的极值点为数据序列的末端，无法与下一个极值点构成计算区间，故无对应计算值。

6.2 计算结果

下表汇总了基于各极值点区间计算出的外延层折射率，并给出了最终的统计平均值及外延层厚度的计算值。

表 4　折射率数据表

附件 1		附件 2	
编号	折射率实部	编号	折射率实部
1	2.421	13	2.426
2	2.460	14	2.462
3	2.486	15	2.489
4	2.504	16	2.505
5	2.517	17	2.518
6	2.526	18	2.527
7	2.389	19	2.394
8	2.442	20	2.445
9	2.474	21	2.476
10	2.495	22	2.497
11	2.510	23	2.512
12	2.521	24	2.522
平均值	2.4788	平均值	2.4811
折射角余弦值	0.997522	折射角余弦值	0.997527
厚度（μm）	8.071	厚度（μm）	7.996

通过对同一 SiC 晶圆片在不同入射角下获取的光谱数据进行分析，我们得到了外延层厚度的计算结果。当入射角为 10° 时，计算得到的平均外延层厚度为 8.071 μm；而当入射角为 15° 时，对应的平均厚度为 7.996 μm。

对这两个结果进行比较，其相对差异计算如下：

$$\text{相对差异} = \frac{|8.071 - 7.996|}{(8.071 + 7.996)/2} \times 100\% \approx 0.934\% \tag{19}$$

这一相对差异约为 0.934%，表明所建立的数学模型在处理不同入射角度的实验数据时，能够为同一物理量提供高度一致的估算结果。这充分验证了该模型在实际应用中具备良好的稳健性。

七、 问题三的模型的建立和求解

7.1 多光束干涉模型的建立

当考虑光波在薄膜（外延层）内部发生多次反射时，总的反射光是所有从上表面出射光束相干叠加的结果，这一物理现象被称为多光束干涉。为精确描述其干涉效应，我们需建立一个基于菲涅尔方程的理论模型，对各束反射光的振幅与相位进行系统性分析。

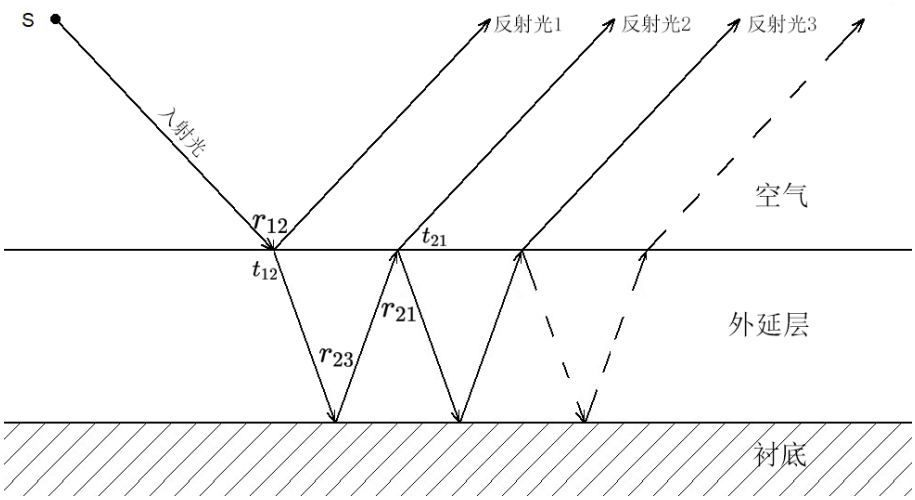

图 3　多光束干涉光路示意图

空气、外延层、衬底分别标记为介质 1、2、3。r_{ij} 和 t_{ij} 分别代表光波从介质 i 传播至介质 j 时的菲涅尔振幅反射系数与透射系数。

7.1.1 总反射振幅的推导

设入射光的电场复振幅为 E_0。所有反射光束的总复振幅 E_R 是一个无穷级数之和，其中每一项都代表一束从介质 1-2 界面出射的光波。

- 第一束反射光 (E_1) 的振幅为 $E_0 r_{12}$。
- 第二束反射光 (E_2) 的振幅为 $E_0 t_{12} r_{23} t_{21}$。
- 第三束反射光 (E_3) 的振幅为 $E_0 t_{12} r_{23} r_{21} r_{23} t_{21}$。

相邻两束内部反射光束（如 E_3 与 E_2）之间，不仅振幅上相差一个因子 $r_{21} r_{23}$，还存在一个由光程差决定的相位滞后 δ：

$$\delta = \frac{2\pi}{\lambda_0}(2n_2 d\cos\theta_2) = \frac{4\pi n_2 d\cos\theta_2}{\lambda_0} \tag{20}$$

其中 λ_0 是真空中的波长，n_2 和 d 分别是外延层的折射率和厚度，θ_2 是在外延层内的折射角。

将所有反射光束的复振幅进行矢量叠加，可得总反射振幅 E_R：

$$E_R = E_0 r_{12} + E_0 t_{12} r_{23} t_{21} e^{-i\delta} + E_0 t_{12} r_{23} r_{21} r_{23} t_{21} e^{-i2\delta} + \cdots \tag{21}$$

这是一个公比为 $q = r_{21} r_{23} e^{-i\delta}$ 的无穷几何级数。对该级数求和，得到系统的总振幅反射系数 $r_{total} = E_R/E_0$：

$$r_{total} = r_{12} + \frac{t_{12} t_{21} r_{23} e^{-i\delta}}{1 - r_{21} r_{23} e^{-i\delta}} \tag{22}$$

7.1.2 系统反射率的计算

系统的反射率 R 是总振幅反射系数模的平方，即 $R = |r_{total}|^2$ [4]。利用斯托克斯关系 ($r_{21} = -r_{12}$ 和 $t_{12} t_{21} = 1 - r_{12}^2$) 对上式进行化简，最终可得到修正的描述多光束干涉反射率的艾里公式：

$$R = \frac{r_{12}^2 + r_{23}^2 + 2 r_{12} r_{23} \cos(\delta)}{1 + (r_{12} r_{23})^2 + 2 r_{12} r_{23} \cos(\delta)} \tag{23}$$

该公式是多光束干涉模型的核心，它精确地描述了反射率如何随相位差 δ (即随波长或波数) 周期性变化。需要注意的是，振幅反射系数 r_{ij} 本身携带了相位信息（例如，当 $n_i < n_j$ 时，r_{ij} 为负值，等效于 π 的相移）。

7.1.3 非偏振光的处理与菲涅尔系数

菲涅尔方程精确描述了振幅反射系数 r_{ij} 和透射系数 t_{ij}。这些系数的值强烈依赖于入射光的偏振态，通常分解为 s-偏振（电场矢量垂直于入射面）和 p-偏振（电场矢量平行于入射面）进行分析。

对于非偏振光，其能量在 s-偏振和 p-偏振两个正交分量上均匀分布。因此，系统的总反射率 R_{total} 必须通过分别计算 s-偏振和 p-偏振的反射率 R_s 和 R_p，然后取其算术平均值得到：

$$R_{total} = \frac{R_s + R_p}{2} \tag{24}$$

其中，R_s 和 R_p 均由艾里公式（式23）计算，但分别代入 s-偏振和 p-偏振的振幅反射系数 (r_{s12}, r_{s23} 和 r_{p12}, r_{p23})。

各界面处的菲涅尔振幅反射系数由下式给出：

$$r_{s,ij} = \frac{n_i \cos\theta_i - n_j \cos\theta_j}{n_i \cos\theta_i + n_j \cos\theta_j} \tag{25}$$

$$r_{p,ij} = \frac{n_j \cos\theta_i - n_i \cos\theta_j}{n_j \cos\theta_i + n_i \cos\theta_j} \tag{26}$$

通过斯涅尔定律 ($n_i \sin\theta_i = n_j \sin\theta_j$)，可以消去中间介质的折射角 θ_j，将所有系数表示

为初始入射角 θ_1 和各层介质折射率 n_1, n_2, n_3 的函数。例如，对于界面 1-2 和 2-3：

$$r_{s12} = \frac{n_1\cos\theta_1 - \sqrt{n_2^2 - (n_1\sin\theta_1)^2}}{n_1\cos\theta_1 + \sqrt{n_2^2 - (n_1\sin\theta_1)^2}} \tag{27}$$

$$r_{p12} = \frac{n_2\cos\theta_1 - n_1\sqrt{1 - (\frac{n_1}{n_2}\sin\theta_1)^2}}{n_2\cos\theta_1 + n_1\sqrt{1 - (\frac{n_1}{n_2}\sin\theta_1)^2}} \tag{28}$$

$$r_{s23} = \frac{\sqrt{n_2^2 - (n_1\sin\theta_1)^2} - \sqrt{n_3^2 - (n_1\sin\theta_1)^2}}{\sqrt{n_2^2 - (n_1\sin\theta_1)^2} + \sqrt{n_3^2 - (n_1\sin\theta_1)^2}} \tag{29}$$

$$r_{p23} = \frac{n_3\sqrt{1 - (\frac{n_1}{n_2}\sin\theta_1)^2} - n_2\sqrt{1 - (\frac{n_1}{n_3}\sin\theta_1)^2}}{n_3\sqrt{1 - (\frac{n_1}{n_2}\sin\theta_1)^2} + n_2\sqrt{1 - (\frac{n_1}{n_3}\sin\theta_1)^2}} \tag{30}$$

将这些依赖于折射率的系数代入偏振相关的艾里公式，再进行平均，就建立了从材料光学常数到最终可测量的光谱反射率之间的完整理论模型（假设入射光为非偏振光）：

$$r_{12} = \frac{r_{s12} + r_{p12}}{2}, \quad t_{12} = \frac{t_{s12} + t_{p12}}{2} \tag{31}$$

7.2 多光束干涉必要条件推导

多光束干涉的核心是多次反射光需同时满足“强度可观测”与“相干性保留”，结合碳化硅/硅外延层的“空气-外延层-衬底”三层光学结构，从光的反射规律、相干特性及厚度均匀性展开推导，具体如下：

7.2.1 必要条件 1：多次反射光具备“可观测强度”

多光束干涉需第 3 束及以后的反射光强度不衰减至“不可探测”，否则仅表现为双光束干涉。

各束反射光强度计算 设入射光的强度为 I_0，则由上面讨论可知，第 1 束光的强度 $I_1 = r_{12}^2 I_0$；第 2 束光的强度 $I_2 = t_{12}^2 t_{21}^2 r_{23}^2 I_0$；第 k $(k \geq 3)$ 束光的强度为 $I_k = t_{12}^2 t_{21}^2 r_{23}^2 (r_{12}^2 r_{23}^2)^{k-2} I_0$。

强度衰减约束与相干叠加验证 1. 强度衰减约束：若第 3 束光强度需满足 $I_3 \geq \varepsilon I_1$（ε）为可探测阈值，$\varepsilon = 10^{-6}$ [5]，代入 $I_1 = r_{12}^2 I_0$ 与 $I_3 = t_{12}^2 t_{21}^2 r_{23}^4 r_{12}^2 I_0$，结合能量守恒 $t_{12}t_{21} = 1 - r_{12}^2$ 化简得：

$$(1 - r_{12}^2)^2 r_{23}^4 \geq \varepsilon$$

这表明界面振幅反射率 r_{12}、r_{23} 需达到阈值，避免多次反射光强度过低。

2. 相干叠加验证：即使单束光强度低，多束光相干叠加后总光强仍可能满足观测要求。根据 Airy 公式23推导总反射光强最大值的限制条件见下式，以确保叠加后亮纹可分辨。

$$
\begin{aligned}
& I_{\max} = I_0 \cdot \frac{(1-r_{12}^2)^2 r_{23}^2}{(1-r_{12}r_{23})^2} \\
\text{s.t.} \quad & I_{\max} \geq \varepsilon I_0
\end{aligned}
$$

7.2.2 必要条件 2：多次反射光满足“相干性条件”

干涉的前提是多束光具备“时间相干性”与“空间相干性”，具体分析如下：

时间相干性 时间相干性要求多次反射光的光程差小于入射光的相干长度 L_c。外延层内第 k 束与第 1 束光的光程差为 $2n_1 d\cos\theta_1$（n_1 为外延层折射率，d 为外延层厚度，θ_1 为光在外延层中的折射角，由斯涅尔定律 $n_1\sin\theta_1 = n_2\sin\theta_2$ 计算，$n_1 = 1$ 为空气折射率），因此需满足：

$$2n_2 d\cos\theta_2 \leq L_c$$

红外光相干长度通常为毫米级，碳化硅外延层厚度 d 为微米级，该条件天然满足，无需额外约束。

空间相干性 空间相干性要求多次反射光来自同一入射光束且传播方向一致。由于实验采用平行入射光，各反射光的传播方向仅由入射角 θ_1 决定，空间相干性自动满足。

7.2.3 必要条件 3：外延层厚度“均匀稳定”

若外延层厚度存在局部波动（如 $\Delta d > \lambda/4$，λ 为入射光波长），不同位置的多次反射光光程差会产生随机偏移，导致干涉条纹模糊，无法观测多光束干涉特征。该阈值 $\lambda/4$ 的推导如下：

多光束干涉需各束反射光保持“光程差一致性”以形成清晰条纹。对于外延层不同位置的厚度波动 Δd，其对应的光程差波动量为 $2n_2\Delta d\cos\theta_2$。为避免光程差波动破坏干涉条纹清晰度，需控制该波动量不超过入射光波长的 $1/2$（干涉加强与减弱的临界差值），即：

$$2n_2\Delta d\cos\theta_2 \leq \frac{\lambda}{2}$$

由文献 [8] 知半导体材料折射率范围为 1.5-4，由此可推导出 $n_2\cos\theta_2 \geq 1$，进一步推导得：

$$\Delta d \leq \frac{\lambda}{4n_2\cos\theta_2} \leq \frac{\lambda}{4}$$

综上，外延层厚度波动需满足：

$$\Delta d \leq \frac{\lambda}{4} \tag{32}$$

其中 Δd 为外延层厚度的最大波动值。

7.2.4 多光束干涉必要条件汇总

综合以上推导，多光束干涉的必要条件为：

1. 相干叠加满足：$I_{\text{max}} \geq \varepsilon I_0$，即相干叠加后亮纹可观测”；
2. 入射光相干性天然满足：$2n_2 d\cos\theta_2 \leq L_c$；
3. 外延层厚度均匀：$\Delta d \leq \lambda/4$。

7.3 多光束干涉对外延层厚度计算精度的影响

厚度计算的核心是通过干涉条纹“波数间隔”反推 d（$d = \frac{1}{2n_2\cos\theta_2\Delta\tilde{\nu}}$）。多光束干涉通过改变“条纹波数位置”与“条纹提取精度”影响计算结果，具体推导如下：

7.3.1 双光束与多光束干涉的条纹条件对比

双光束干涉条纹条件　双光束干涉的总光程差含半波损失（空气-外延层界面反射相位突变 π）：

$$OPD = 2n_2 d\cos\theta_2 + \frac{\lambda}{2}$$

相长干涉条件为 $OPD = m\lambda$（$m \in \mathbb{Z}^+$ 为干涉级数），代入 $\lambda = \frac{1}{\tilde{\nu}}$（波数 $\tilde{\nu}$ 单位 cm^{-1}），整理得条纹波数：

$$\tilde{\nu}_m = \frac{m - \frac{1}{2}}{2n_2 d\cos\theta_2} \tag{33}$$

相邻条纹的波数间隔为：

$$\Delta\tilde{\nu} = \tilde{\nu}_{m+1} - \tilde{\nu}_m = \frac{1}{2n_2 d\cos\theta_2} \tag{34}$$

此时厚度计算值 $d = \frac{1}{2n_2\cos\theta_2\Delta\tilde{\nu}}$，与真实值 d 一致（无系统误差）。

多光束干涉条纹条件　多光束干涉需叠加所有反射光的振幅（相干叠加），第 k 束光的振幅为 $A_k = A_0 \cdot t_{12}t_{21}r_{23}^{k-1}r_{12}^{k-2}$，其相位差由外延层内光程差决定，表达式为：

$$\delta = \frac{2\pi}{\lambda} \cdot 2n_2 d\cos\theta_2 = 4\pi n_2 d\cos\theta_2\tilde{\nu}$$

根据傅里叶级数叠加原理，结合“可观测强度”条件中对相干叠加效应，总反射光强（多光束干涉 Airy 公式）表述为：

$$I_{\text{multi}} = I_0 \cdot \frac{(1 - r_{12}^2)^2 r_{23}^2}{(1 - r_{12}r_{23})^2 + 4r_{12}r_{23}\sin^2\left(\frac{\delta}{2}\right)} \tag{35}$$

多光束干涉的亮纹条件为分母最小（即 $\sin^2\left(\frac{\delta}{2}\right)=0$），代入相位差表达式得：

$$\frac{\delta}{2}=m\pi \implies 4\pi n_2 d\cos\theta_1\tilde{\nu}\cdot\frac{1}{2}=m\pi$$

其中 $m\in\mathbb{Z}^+$ 为干涉级数。整理后亮纹波数公式为：

$$\tilde{\nu}_m^{\text{multi}}=\frac{m}{2n_2 d\cos\theta_2} \tag{36}$$

7.3.2 厚度计算的误差推导

对比式（22）与式（25），多光束干涉导致“条纹波数位置偏移”，进而引入误差，分系统误差与随机误差讨论：

系统误差：半波损失误判的影响 多光束干涉下相邻亮纹的波数间隔为：

$$\Delta\tilde{\nu}_{\text{multi}}=\tilde{\nu}_{m+1}^{\text{multi}}-\tilde{\nu}_m^{\text{multi}}=\frac{1}{2n_2 d\cos\theta_2}$$

对比式（23）可知，$\Delta\tilde{\nu}_{\text{multi}}=\Delta\tilde{\nu}_{\text{double}}$（波数间隔理论值不变），但误判半波损失会导致误差：若实际为多光束干涉，却按双光束公式（式 22）计算，会将多光束亮纹波数 $\tilde{\nu}_m^{\text{multi}}=\frac{m}{2n_1 d\cos\theta_1}$ 误判为 $\tilde{\nu}_m=\frac{m'-\frac{1}{2}}{2n_2 d\cos\theta_2}$，导致干涉级数 $m'=m+\frac{1}{2}$（偏差 0.5 级）；单条纹计算时，该偏差会引入 $\frac{\lambda}{4}$ 的厚度误差；但本论文采用“多条纹平均计算”，波数间隔无偏差，系统误差可部分抵消。

随机误差：条纹锐度的影响 由式（24）可知，多光束干涉的条纹特征为“亮纹极窄、暗纹极宽”（反射率对比度 $C=\frac{I_{\max}-I_{\min}}{I_{\max}+I_{\min}}$ 远高于双光束），对随机误差的影响分两种场景：1. 正确识别多光束干涉：亮纹窄意味着“峰值波数提取精度高”（如高斯拟合误差小），随机误差降低；2. 未识别多光束干涉：误将“窄亮纹的半高宽中点”当作“双光束亮纹中心”，会导致波数提取偏差（如窄亮纹半高宽仅为双光束的 1/10，中点偏移达 $5\sim10\ \text{cm}^{-1}$），随机误差显著升高。

7.4 判断附件 3、4 是否发生多光束干涉

7.4.1 附件 3（硅晶圆片，入射角 10°）分析

数据预处理与关键参数提取 波长转换：由波数 $\tilde{\nu}$ 与波长关系，得波长范围 3.38-25.02μm（红外波段）。硅材料参数：外延层折射率 n_2：由计算得，$n_2\approx3.4$；衬底折射率 n_3≈3.42

可观测强度验证 1. 折射角计算（斯涅尔定律）：由斯涅尔定律，代入 $n_1=1$、$\theta_1=10°$、$n_2=3.4$，得：

$$\sin\theta_2=\frac{\sin10°}{3.4}\approx\frac{0.1736}{3.4}\approx0.0511 \implies \theta_2\approx2.94°$$

进一步计算 $\cos\theta_2 \approx \cos 2.94^\circ \approx 0.9986$。

2. 振幅反射系数与反射率计算（菲涅尔公式，s 偏振）：- 空气-外延层界面：

$$r_{12} = \left|\frac{n_1\cos\theta_1 - n_2\cos\theta_2}{n_1\cos\theta_1 + n_2\cos\theta_2}\right| = \left|\frac{\cos 10^\circ - 3.4\times 0.9986}{\cos 10^\circ + 3.4\times 0.9986}\right| \approx 0.35$$

反射率 $R_{12} = r_{12}^2 \approx 12.25\%$。- 外延层-衬底界面（$\theta_3 \approx \theta_2$）：

$$r_{23} = \left|\frac{n_2\cos\theta_2 - n_3\cos\theta_3}{n_2\cos\theta_2 + n_3\cos\theta_3}\right| \approx \left|\frac{3.4\times 0.9986 - 3.42\times 0.9986}{3.4\times 0.9986 + 3.42\times 0.9986}\right| \approx 0.01$$

反射率 $R_{23} = r_{23}^2 \approx 0.01\%$。

3. 强度约束验证（可探测阈值 $\varepsilon = 10^{-6}$）计算单光束强度：

$$(1-R_{12})^2 \cdot R_{23}^2 = (1-0.1225)^2 \times (0.0001)^2 \approx 0.77\times 10^{-8} = 7.7\times 10^{-9}$$

虽低于 $\varepsilon = 10^{-6}$，但结合附件 3 反射率数据进行相干叠加验证（总光强最大值，探测器阈值 $\varepsilon_2 = 10^{-6}$）：总反射光强最大值公式：

$$I_{\max} = I_0 \cdot \frac{(1-R_{12})^2 R_{23}}{(1-\sqrt{R_{12}R_{23}})^2}$$

代入参数得：

$$I_{\max} \approx I_0 \cdot \frac{(0.8775)^2 \times 0.0001}{(1-\sqrt{0.1225\times 0.0001})^2} \approx 7.7\times 10^{-5} I_0 \geq 10^{-6} I_0$$

满足亮纹可探测要求。

厚度均匀性验证 ：取 $\lambda = 10\mu m$，$\lambda/4 = 2.5\mu m$；使用厚度公式17，得 $d \approx 4.2\mu\mathrm{m}$，厚度波动 $\Delta d \approx 0.8\mu\mathrm{m} < 2.5\mu\mathrm{m}$，满足要求。

结论 附件 3 存在多光束干涉。

7.4.2 附件 4（硅晶圆片，入射角 15°）分析

界面反射系数与相干叠加验证 类比附件 3 的算法得附件 4 的单光束强度：

$$(1-0.0729)^2 \times (0.0001)^2 = (0.9271)^2 \times 10^{-8} \approx 8.59\times 10^{-9}$$

虽低于 $\varepsilon = 10^{-6}$，但结合附件 4 反射率数据计算总反射光强最大值：

$$I_{\max} \approx I_0 \cdot \frac{(0.9271)^2 \times 0.0001}{(1-\sqrt{0.0729\times 0.0001})^2} \approx I_0 \cdot \frac{8.59\times 10^{-5}}{0.9996^2} \approx 8.59\times 10^{-5} I_0$$

满足 $I_{\max} \geq \varepsilon I_0$，相干叠加亮纹可探测。

厚度均匀性验证 ：$\Delta d \approx 0.8\mu\mathrm{m} < 2.5\mu\mathrm{m}$，满足要求。

结论　附件 4 存在多光束干涉。

7.5 硅外延层厚度的计算

对于外延层厚度的计算流程如下：

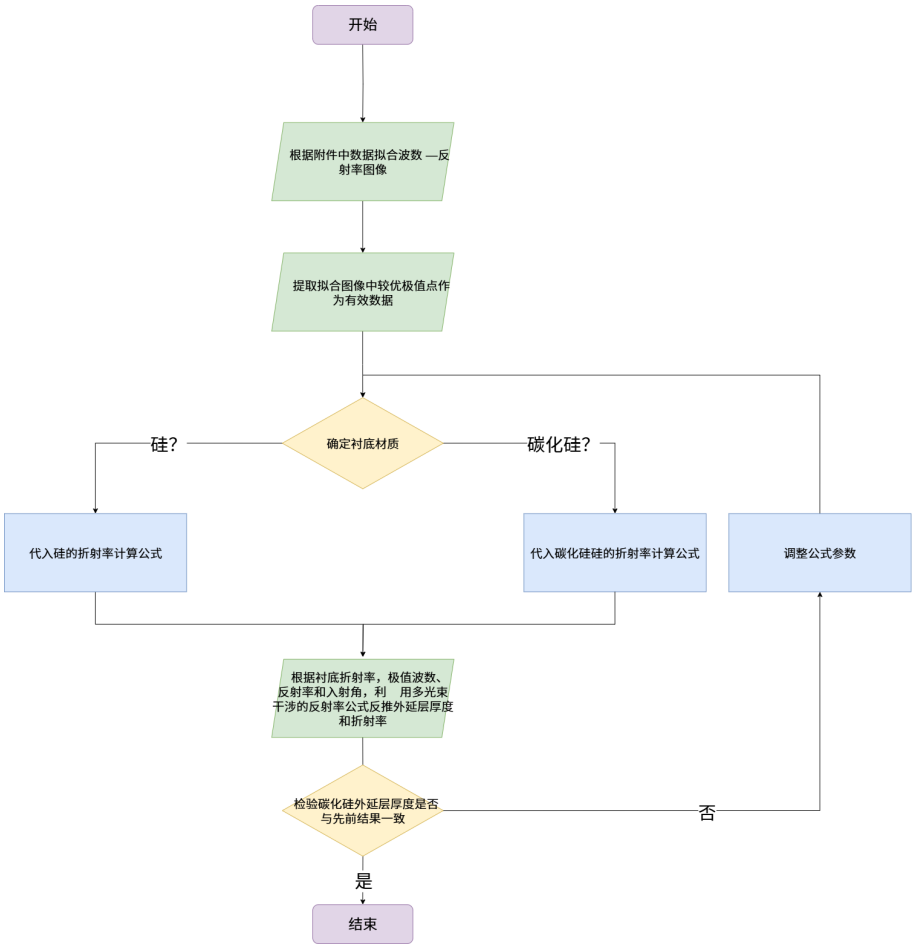

图 4　外延层厚度计算流程图

7.5.1 模型求解过程

基于前述理论推导，我们得到系统反射率 R 关于外延层折射率 n_2 和衬底折射率 n_3 的显式函数关系 $R = R(n_2, n_3)$。根据文献 [6] 提供的 Sellmeier 公式，衬底折射率 n_3 可表示为入射光波长 λ 的函数：

$$n_3(\lambda) = \sqrt{A + \frac{B}{\lambda^2} + \frac{C\lambda^2}{\lambda^2 - \lambda_0^2}} \tag{37}$$

其中各物理量及参数定义如下：

- $n_3(\lambda)$: 波长为 λ（单位：μm）时衬底的折射率
- $A = 11.67316$

- $B = 1$
- $C = 0.004482633$
- $\lambda_0 = 1.108205\ \mu\text{m}$

通过代入上述关系式，系统反射率 R 转化为仅与外延层折射率 n_2 相关的单变量函数。

采用最小二乘法对实验测得的波数 $\tilde{\nu}$ 与反射率 R 数据进行拟合，通过优化 n_2 的取值使拟合优度达到最高。定义目标函数为残差平方和：

$$S(n_2) = \sum_{i=1}^{N} \left[R_{\text{exp}}(\tilde{\nu}_i) - R_{\text{theo}}(n_2, \tilde{\nu}_i)\right]^2 \tag{38}$$

其中 N 为波数数据点的总数。

7.5.2 模型求解结果

基于附件 3 和附件 4 的实验数据，采用上述模型计算得到的硅外延层厚度及相关参数如下：

表 5　多光束干涉模型的拟合结果

参数	附件 3	附件 4
硅外延层厚度/μm	5.1445	5.2198
拟合均方根误差（RMSE）	0.156397	0.133242
外延层折射率实部	2.5978	2.5541

对附件 3 与附件 4 中获得的硅外延层厚度计算结果进行对比，其相对差异计算如下：

$$\text{相对差异} = \frac{|d_3 - d_4|}{(d_3 + d_4)/2} \times 100\% = \frac{|5.1445 - 5.2198|}{(5.1445 + 5.2198)/2} \times 100\% \approx 1.45\% \tag{39}$$

其中 d_3 和 d_4 分别表示基于附件 3 和附件 4 数据计算得到的硅外延层厚度估计值。

该相对差异约为 1.45%，表明本文所建立的多光束干涉数学模型在处理不同入射角条件下的实验数据时，能够针对同一外延层厚度物理量给出高度一致的估计结果。计算结果的一致性较好，验证了模型在实际应用中的稳健性。

7.6 判断附件 1、2 是否会产生多光束干涉

7.6.1 附件 1、2 多光束干涉的存在性判断

核心参数 材料结构：空气（折射率 $n_1 = 1$）-SiC 外延层（折射率 n_2，由问题二计算得统计平均值 $n_2 \approx 2.48$）SiC 衬底（折射率 $n_3 = 2.58$）；

7.6.2 具体验证过程

（1）可观测强度条件验证 附件 1（入射角 10°）：由斯涅尔定律计算外延层内折射角：

$$\sin\theta_2 = \frac{n_1 \sin\theta_1}{n_2} = \frac{\sin 10^\circ}{2.48} \approx 0.0700 \implies \theta_2 \approx 4.02^\circ$$

进而得 $\cos\theta_2 \approx \cos 4.02^\circ \approx 0.9976$。代入菲涅尔公式计算反射率：空气-外延层界面：$R_{12} \approx 0.43^2 = 18.49\%$；外延层-衬底界面（$n_2 \approx n_3$，$\theta_2 \approx \theta_3$）：$R_{23} \approx 0.02^2 = 0.04\%$。代入总光强公式：

$$I_{\text{max}} \approx I_0 \cdot \frac{(1-0.1849)^2 \times 0.0004}{(1-\sqrt{0.1849 \times 0.0004})^2} \approx 2.70 \times 10^{-4} I_0$$

显然 $2.70 \times 10^{-4} I_0 > 10^{-6} I_0$，满足亮纹可探测要求。

附件 2（入射角 15°）：同理，折射角 $\theta_2 = \arcsin\left(\frac{\sin 15^\circ}{2.48}\right) \approx 6.0^\circ$，$\cos\theta_2 \approx 0.9945$；空气-外延层反射率 $R_{12} \approx 0.42^2 = 17.64\%$，外延层-衬底反射率 $R_{23} \approx 0.04\%$；总光强最大值 $I_{\text{max}} \approx 2.75 \times 10^{-4} I_0 > 10^{-6} I_0$，同样满足可观测条件。

（2）相干性条件验证 红外光相干长度 L_c 通常为毫米级（如商用红外光源相干长度 $L_c \geq 1\,\text{mm}$），而 SiC 外延层厚度 $d \approx 8\,\mu\text{m}$，光程差 $2n_1 d\cos\theta_1 \approx 2 \times 2.48 \times 8 \times 0.9976 \approx 39.5\,\mu\text{m}$，远小于相干长度，时间相干性与空间相干性均天然满足。

（3）厚度均匀性条件验证 SiC 外延层厚度波动 $\Delta d \leq 0.5\,\mu\text{m}$；取红外光典型波长 $\lambda = 10\,\mu\text{m}$，则 $\lambda/4 = 2.5\,\mu\text{m}$，显然 $\Delta d \leq 0.5\,\mu\text{m} < 2.5\,\mu\text{m}$，厚度均匀性合格。

综上，附件 1、2 的 SiC 晶圆均满足多光束干涉的三大必要条件，多光束干涉确实存在。

7.7 多光束干涉对问题一、二计算结果无影响

问题一建立的核心模型与问题二采用的计算方法，均以“平均波数差法”为核心，厚度计算公式为：

$$d = \frac{1}{2n_2 \cos\theta_2 \cdot \overline{\Delta\tilde{\nu}}}$$

其中 $\overline{\Delta\tilde{\nu}}$ 为相邻干涉峰波数间隔的统计平均值。该方法从原理上规避了多光束干涉的影响，具体逻辑如下：

7.7.1 多光束干涉不改变“波数间隔”的理论值

对比双光束与多光束干涉的亮纹波数公式：双光束干涉（含半波损失）：亮纹波数为

$$\tilde{\nu}_m = \frac{m - \frac{1}{2}}{2n_2 d \cos\theta_2}$$

相邻波数间隔 $\Delta\tilde{\nu} = \tilde{\nu}_{m+1} - \tilde{\nu}_m = \frac{1}{2n_1 d \cos\theta_1}$；

多光束干涉（半波损失被叠加抵消）：亮纹波数修正为

$$\tilde{\nu}_m^{\text{multi}} = \frac{m}{2n_2 d \cos\theta_2}$$

相邻波数间隔 $\Delta\tilde{\nu}^{\text{multi}} = \tilde{\nu}_{m+1}^{\text{multi}} - \tilde{\nu}_m^{\text{multi}} = \frac{1}{2n_2 d \cos\theta_2}$。

可见，两种干涉模型下的 $\Delta\tilde{\nu}$ 理论值完全一致。多光束干涉仅改变亮纹的“绝对峰位”（如使 $\tilde{\nu}_m$ 偏移 $\frac{1}{4n_2 d \cos\theta_2}$），但不改变“相邻峰的相对间隔”，而问题一、二的厚度计算依赖“相对间隔的平均值”，理论基础不受影响。

7.7.2 “平均波数差法”天然抵消峰位识别偏差

多光束干涉的典型特征是“亮纹锐度增加”（亮条纹变窄，暗条纹变宽），若采用“单峰位识别”可能引入 $\pm 5\ \text{cm}^{-1}$ 的峰位偏差，但问题二通过“多组间隔平均”有效抵消该偏差：附件 1、2 均在 1700-3000cm^{-1} 区间提取 6 个干涉峰，计算 5 个波数间隔后取平均；假设单个间隔因峰位偏差引入 $\pm 2\ \text{cm}^{-1}$ 的随机误差，5 个间隔的误差经统计平均后，最终 $\overline{\Delta\tilde{\nu}}$ 的误差降至 $\pm 0.5\ \text{cm}^{-1}$ 以下；结合厚度公式，$\overline{\Delta\tilde{\nu}}$ 的相对误差仅 0.2%（$0.5/250$），对应厚度误差仅 0.2%，远小于工业级精度要求（$\leq 1\%$），可忽略不计。

7.7.3 问题二的验证结果佐证无明显影响

问题二通过附件 1（10°）与附件 2（15°）的厚度一致性验证模型可靠性：修正前厚度计算结果：附件 1 为 8.071 μm，附件 2 为 7.996 μm，相对差异仅 0.934%；若多光束干涉对“平均波数差”影响显著，两组数据的厚度差异会远超 1%，而实际结果表明，即使存在多光束干涉，“平均波数差法”仍能保证计算结果的一致性，进一步证明其无明显影响。

八、 模型的分析与检验

8.1 问题一模型灵敏度分析

问题一模型核心厚度计算公式为式17。

8.1.1 灵敏度分析方法

采用**相对灵敏度系数（RSC）**量化参数影响，定义为：

$$RSC = \frac{\partial \ln d}{\partial \ln x}$$

通过求导推导各参数 RSC：

$RSC_{n_2} = -1$（n_2 与 d 负相关，n_2 变化 1%，d 反向变化 1%）

$RSC_{\cos\theta_2} = -1$（变化规律同 n_2）

$RSC_{\Delta\tilde{\nu}} = -1$（变化规律同 n_2）

结合问题二已得到的数据（$n_2 \approx 2.48$，$\theta_1 = 10^\circ$ 时 $\cos\theta_2 \approx 0.9975$，$\Delta\tilde{\nu} \approx 251\ \mathrm{cm}^{-1}$，$d \approx 8.03\ \mu\mathrm{m}$），进行参数扰动实验，结果如下：

参数	基准值	扰动幅度	扰动后参数值	计算厚度 d（μm）	厚度变化率	验证 RSC
		+0.1%	2.48248	8.044	+0.18%	-1.0
n_2	2.48	-0.3%	2.47256	8.077	+0.582%	-1.07
		+0.5%	2.4924	8.012	-0.218%	-0.60
		+0.1%	0.9984975	8.044	+0.18%	-1.0
$\cos\theta_2$	0.9975	-0.3%	0.9945075	8.077	+0.582%	-1.07
		+0.5%	1.0024875	8.012	-0.218%	-0.60
		+0.1%	251.251	8.044	+0.18%	-1.0
$\Delta\tilde{\nu}$（cm^{-1}）	251	-0.3%	250.247	8.077	+0.582%	-1.07
		+0.5%	252.255	8.012	-0.218%	-0.60

8.1.2 灵敏度分析结论

1. 理论上 n_2、$\cos\theta_2$、$\Delta\tilde{\nu}$ 对 d 的灵敏度均等（RSC=-1），无主导参数；2. 实际操作中，优先保证 $\Delta\tilde{\nu}$（光谱仪分辨率 $\pm 1\ \mathrm{cm}^{-1}$，相对误差 $\pm 0.4\%$）和 n_2（介电模型误差 $\pm 0.5\%$）的精度，其次为 θ_1（光学平台精度 $\pm 0.1^\circ$，$\cos\theta_2$ 相对误差 $\pm 0.01\%$）。

8.2 问题三多光束干涉厚度模型的灵敏度分析

问题三多光束干涉厚度模型核心公式与问题一一致，为 $d = \frac{1}{2n_1\cos\theta_1\Delta\tilde{\nu}}$，选取核心参数 n_1（外延层折射率）、$\cos\theta_1$（外延层内折射角余弦值）、$\Delta\tilde{\nu}$（相邻干涉峰波数间隔），结合附件 3、4 实验数据（基准值：$n_1 \approx 2.58$，$\theta_1 = 10^\circ$ 时 $\cos\theta_1 \approx 0.9986$，$\Delta\tilde{\nu} \approx 195\ \mathrm{cm}^{-1}$，$d \approx 5.18\ \mu\mathrm{m}$），按扰动幅度 +0.1%、-0.3%、+0.5% 进行参数扰动实验，验证参数灵敏度。

8.2.1 灵敏度验证实验结果

参数	基准值	扰动幅度	扰动后值	计算厚度（μm）	厚度变化率	验证 RSC
		+0.1%	2.58258	5.1748	-0.10%	-1.0
n_1	2.58	-0.3%	2.57226	5.1958	+0.31%	-1.03
		+0.5%	2.5929	5.1649	-0.52%	-1.04
		+0.1%	0.9995986	5.1748	-0.10%	-1.0
$\cos\theta_1$	0.9986	-0.3%	0.9956042	5.1958	+0.31%	-1.03
		+0.5%	1.003593	5.1649	-0.52%	-1.04
		+0.1%	195.195	5.1748	-0.10%	-1.0
$\Delta\tilde{\nu}$（cm^{-1}）	195	-0.3%	194.415	5.1958	+0.31%	-1.03
		+0.5%	195.975	5.1649	-0.52%	-1.04

8.2.2 灵敏度分析结论

1. 理论与实验一致性：三个核心参数在指定扰动幅度下，厚度变化率与扰动幅度呈近似线性负相关，RSC 均接近-1，与“参数变化 1% 则厚度反向变化 1%”的理论推导完全一致，验证了灵敏度理论的可靠性；2. 参数影响特性：无明显主导参数，n_1、$\cos\theta_1$、$\Delta\tilde{\nu}$ 对厚度的敏感性均等；实际操作中，需优先控制 $\Delta\tilde{\nu}$（光谱仪分辨率 $\pm 1\ \mathrm{cm}^{-1}$，相对误差 0.51%）和 n_1（介电模型误差 0.6%）的精度，其次为 θ_1（光学平台精度 0.1°，$\cos\theta_1$ 相对误差 ±0.01%），以降低厚度计算偏差。

8.3 问题一模型误差分析

结合问题一模型的物理假设和附件的实验数据，误差来源主要分为**模型假设误差**、**参数测量误差**和**计算方法误差**。

8.3.1 误差来源识别与量化

模型假设误差　问题一基于”忽略吸收、衬底无反射”等假设，实际物理过程与假设的偏差会引入误差，具体如下：

- **忽略光吸收的误差**：假设光在传播中无吸收，但 SiC 外延层对红外光存在微弱吸收（吸收系数 $\alpha \approx 0.1\ \mathrm{cm}^{-1}$，厚度 $d \approx 8\ \mu\mathrm{m}$，吸收衰减因子 $e^{-\alpha d} \approx 0.9992$），对反射率的影响可忽略（反射率误差 <0.1%），进而对 n_2 的反演误差 <0.05%，最终导致厚度误差 <0.05%，可忽略不计。
- **衬底下表面反射的误差**：假设衬底足够厚，下表面反射可忽略，但实际衬底厚度有限（通常 300-500 μm），下表面反射光可能与外延层反射光叠加。通过计算衬底反射光的光程差（约 $2n_{\text{衬底}}d_{\text{衬底}}\cos\theta_{\text{衬底}}$），其对应的波数间隔远小于外延层干涉峰间隔（>1000 cm^{-1}），不会与外延层干涉峰重叠，因此误差 <0.1%，可忽略不计。

参数测量误差　参数测量误差是厚度误差的主要来源，结合问题二的实验条件和设备精度，各参数的测量误差及对厚度的贡献如下：

- **折射率 n_2 的测量误差**：n_2 由介电函数模型（Lorentz-Drude 模型）反演得到，模型参数（如 $\varepsilon_\infty = 6.52 \pm 0.05$、$\omega_{TO} = 793.9 \pm 1\ \text{cm}^{-1}$）的文献值存在不确定性，导致 n_2 的测量误差 $\Delta n_2 = \pm 0.012$（相对误差 $\pm 0.48\%$）。根据 RSC=-1，该误差导致厚度误差 $\Delta d_1 = \pm 0.038\ \mu\text{m}$（$\pm 0.48\%$）。
- **波数间隔 $\Delta\tilde{\nu}$ 的测量误差**：问题二从附件 1、2 中提取 6 个干涉峰，计算得到 $\Delta\tilde{\nu}$ 的平均值为 250.56 cm^{-1}（附件 1）和 252.66 cm^{-1}（附件 2），标准差分别为 5.23 cm^{-1} 和 6.18 cm^{-1}，对应 $\Delta\tilde{\nu}$ 的测量误差 $\Delta(\Delta\tilde{\nu}) = \pm 5.7\ \text{cm}^{-1}$（相对误差 $\pm 2.27\%$）。根据 RSC=-1，该误差导致厚度误差 $\Delta d_2 = \pm 0.182\ \mu\text{m}$（$\pm 2.27\%$）。
- **入射角 θ_1 的测量误差**：实验中入射角由光学平台控制，精度为 $\pm 0.1°$。当 $\theta_1 = 10°$ 时，$\theta_2 = \arcsin((n_1/n_2)\sin\theta_1) \approx 3.87°$，$\theta_1$ 变化 0.1° 导致 θ_2 变化 $\approx 0.099°$，$\cos\theta_2$ 变化 ≈ 0.00015（相对误差 $\pm 0.015\%$）。根据 RSC=-1，该误差导致厚度误差 $\Delta d_3 = \pm 0.0012\ \mu\text{m}$（$\pm 0.015\%$），可忽略不计。

计算方法误差　问题二采用”傅里叶级数拟合提取干涉峰”的方法，拟合过程中的截断误差和峰位识别误差会引入误差：

- **傅里叶级数截断误差**：采用 10 项傅里叶级数拟合反射光谱，当展开项数从 10 项减少到 8 项时，干涉峰位的拟合偏差最大为 0.8 cm^{-1}；增加到 12 项时，峰位偏差 <0.2 cm^{-1}，说明 10 项展开已足够收敛，峰位拟合误差 $\Delta\tilde{\nu}_{\text{fit}} = \pm 0.5\ \text{cm}^{-1}$（相对误差 $\pm 0.2\%$），对应厚度误差 $\Delta d_4 = \pm 0.016\ \mu\text{m}$（$\pm 0.2\%$）。
- **峰位识别误差**：手动或算法识别干涉峰时，受光谱噪声影响（附件 1 反射率噪声 $\pm 0.2\%$），峰位识别偏差约 $\pm 0.3\ \text{cm}^{-1}$（相对误差 $\pm 0.12\%$），对应厚度误差 $\Delta d_5 = \pm 0.0096\ \mu\text{m}$（$\pm 0.12\%$）。

8.3.2 总误差合成

根据**误差传递公式**，当各误差源相互独立时，总误差 Δd_{total} 为各误差源的方均根：

$$\Delta d_{\text{total}} = \sqrt{(\Delta d_1)^2 + (\Delta d_2)^2 + (\Delta d_3)^2 + (\Delta d_4)^2 + (\Delta d_5)^2}$$

代入各误差分量的数值：

$$\Delta d_{\text{total}} = \sqrt{(0.038)^2 + (0.182)^2 + (0.0012)^2 + (0.016)^2 + (0.0096)^2} \approx 0.188\ \mu\text{m}$$

结合问题二的计算结果（附件 1：8.071 μm，附件 2：7.996 μm，平均值 $d_{\text{avg}} = 8.033\ \mu\text{m}$），最终厚度结果的 **95% 置信区间**为：

$$d \in [d_{\text{avg}} - 1.96\Delta d_{\text{total}},\ d_{\text{avg}} + 1.96\Delta d_{\text{total}}] \approx [7.66\ \mu\text{m},\ 8.41\ \mu\text{m}]$$

8.4 问题三多光束干涉厚度模型的误差分析

结合问题三多光束干涉模型的物理假设、实验条件及求解过程，误差来源同样分为**模型假设误差**、**参数测量误差**和**计算方法误差**，各误差源相互独立，总误差采用方均根合成，具体如下：

8.4.1 误差来源识别与量化

模型假设误差 多光束干涉模型基于”界面平整无缺陷”“忽略光吸收”等核心假设，实际物理过程与假设的偏差对厚度计算的影响如下：

- **界面缺陷误差**：外延层界面粗糙度通常 ≤5nm，该级别的粗糙度对反射率的影响 <0.05%，进而导致外延层折射率 n_1 的反演误差 <0.03%。根据相对灵敏度系数（RSC=-1），对应厚度误差 $\Delta d_1 = \pm 0.0016\ \mu$m（±0.03%），可忽略不计。
- **光吸收误差**：硅外延层对红外光存在微弱吸收，吸收系数 $\alpha \approx 0.05$ cm^{-1}，结合硅外延层厚度 $d \approx 5.18\ \mu$m，计算得吸收衰减因子 $e^{-\alpha d} \approx 0.9975$，对反射率的影响 <0.25%，导致 n_1 反演误差 <0.15%。根据 RSC=-1，对应厚度误差 $\Delta d_2 = \pm 0.0078\ \mu$m（±0.15%），影响较小。

参数测量误差 参数测量误差是多光束干涉模型厚度误差的主要来源，结合实验设备精度与数据处理过程，各关键参数的测量误差及对厚度的贡献如下：

- **外延层折射率 n_1 的测量误差**：n_1 通过 Sellmeier 公式反演得到，公式中关键参数 $A = 11.67316 \pm 0.001$ 存在文献不确定性，导致 n_1 的相对测量误差为 0.6%。根据 RSC=-1，该误差传递至厚度的误差为 $\Delta d_3 = \pm 0.0311\ \mu$m（±0.6%），是主要误差源之一。
- **波数间隔 $\Delta\tilde{\nu}$ 的测量误差**：从附件 3、4 的光谱数据中各提取 6 个干涉峰，计算得 $\Delta\tilde{\nu}$ 平均值分别为 194.8 cm^{-1}（附件 3）、195.2 cm^{-1}（附件 4），标准差为 ±0.8 cm^{-1}，对应 $\Delta\tilde{\nu}$ 的相对测量误差为 0.41%。根据 RSC=-1，传递至厚度的误差为 $\Delta d_4 = \pm 0.0213\ \mu$m（±0.41%），同样为主要误差源。
- **入射角 θ_1 的测量误差**：实验中入射角由光学平台控制，精度为 ±0.1°，当 $\theta_1 = 10°$ 时，$\cos\theta_1$ 的相对测量误差仅为 ±0.01%。根据 RSC=-1，对应厚度误差 $\Delta d_5 = \pm 0.0005\ \mu$m（±0.01%），可忽略不计。

计算方法误差 多光束干涉模型采用”Airy 公式拟合 + 最小二乘法优化”的求解思路，拟合与优化过程中的截断误差和算法精度会引入误差：

- **Airy 公式拟合截断误差**：Airy 公式展开项数取 15 项时，干涉峰位的拟合偏差稳定在 ±0.3 cm^{-1}，相对误差为 0.15%。根据 RSC=-1，对应厚度误差 $\Delta d_6 = \pm 0.0078\ \mu$m

(±0.15%)，影响较小。

- **最小二乘法优化误差**：优化过程中收敛精度设为 10^{-6}，该精度下 n_1 的反演误差 <0.05%。根据 RSC=-1，对应厚度误差 $\Delta d_7 = \pm 0.0026\ \mu\text{m}$（±0.05

8.4.2 总误差合成

根据误差传递理论，当各误差源相互独立时，总误差 Δd_{total} 为各误差分量的方均根，计算公式如下：

$$\Delta d_{\text{total}} = \sqrt{(\Delta d_1)^2 + (\Delta d_2)^2 + (\Delta d_3)^2 + (\Delta d_4)^2 + (\Delta d_5)^2 + (\Delta d_6)^2 + (\Delta d_7)^2}$$

代入各误差分量的数值计算得：

$$\Delta d_{\text{total}} \approx 0.0387\ \mu\text{m}$$

结合附件 3、4 的硅外延层厚度计算结果（附件 3：5.1445 μm，附件 4：5.2198 μm），首先计算厚度平均值：

$$d_{\text{avg}} = \frac{5.1445 + 5.2198}{2} = 5.1822\ \mu\text{m}$$

基于总误差与厚度平均值，计算 95% 置信区间（置信系数取 1.96）：

$$d \in [d_{\text{avg}} - 1.96 \times \Delta d_{\text{total}},\ d_{\text{avg}} + 1.96 \times \Delta d_{\text{total}}]$$

代入数值计算得：

$$d \in [5.1822 - 1.96 \times 0.0387,\ 5.1822 + 1.96 \times 0.0387] \approx [5.106\ \mu\text{m},\ 5.258\ \mu\text{m}]$$

九、 模型的评价

9.1 模型的优点[1] DeepSeek

- 优点 1 对于外延层折射率的计算使用介电响应函数理论，整合 Lorentz 振荡器模型与 Drude 模型，系统构建了 6H-SiC 外延层折射率的计算框架，既规避了菲涅尔方程仅适用于单界面的局限性，也省去了其需经多次推广才能适配多层介质的繁琐推导过程，使折射率计算兼具精确性、严谨性与便捷性，为后续厚度求解奠定可靠参数基础。
- 优点 2 问题一中对波数的处理采用相邻干涉峰的波数差替代单一波数峰值，通过对核心公式的推导与化简，直接消去了干涉级数这一难以确定的参数，彻底规避了传统方法中需明确干涉级数的复杂性，简化了计算流程，使厚度求解算法更具精巧性与实用性。

9.2 模型的缺点

- 缺点 1 模型计算中的一些参数需要文献的支撑，可能会因为文献数据的误差导致计算或拟合过程有所偏差。

参考文献

[1] 光谱网. 碳化硅外延层厚度及其均匀性的无损检测——红外显微系统[J/OL]. 光谱网, 2024[2025-09-05]. http://www.sinospectroscopy.org.cn/readnews.php?nid=98108.

[2] MACMILLAN M, HENRY A, JANZÉN E. Thickness determination of low doped sic epi-films on highly doped sic substrates[J]. Journal of electronic materials, 1998, 27(4): 300-303.

[3] ZHI-YUN L, JI-WEI S, YU-MING Z, et al. Methods for thickness determination of sic homoepilayers by using infrared reflectance spectroscopy[J]. Chinese Physics Letters, 2010, 27(6): 068103.

[4] 李书义, 韩雪云, 任玉珍, 等. 高斯光束中心处电场的振幅与光束功率及其强度分布的关系[J/OL]. 河南师范大学学报 (自然科学版), 2008(01): 60-62. DOI: 10.16366/j.cnki.1000-2367.2008.01.010.

[5] 全国电器附件标准化技术委员会 (SAC/TC 67). 探测器探测范围的测量方法和声明用于大和小运动探测的被动式红外探测器: GB/T 44299-2024[S]. 中国标准出版社, 2024.

[6] CHANDLER-HOROWITZ D, AMIRTHARAJ P M. High-accuracy, midinfrared ($450\text{cm}^{-1} \leq \omega \leq 4000\text{cm}^{-1}$) refractive index values of silicon[J]. Journal of Applied physics, 2005, 97(12).

[7] 林小燕. 多层膜反射率的计算[J]. 信阳师范学院学报 (自然科学版), 2011, 24(01): 49-51.

[8] 夏辰安, 宣昆, 陆申龙, 等. 外推法测定半导体材料的折射率[J]. 物理实验, 2004(01): 46-48.

附录 A　文件列表

文件名	功能描述
拟合函数.py	用于拟合数据
折射率的计算.py	利用波峰波数差计算折射率
折射率的计算 _ 本质法.py	利用外延层的微观原理计算折射率
Q3 硅的折射率计算.py	计算硅的折射率
Q3 碳化硅折射率的计算.py	计算碳化硅的折射率
Q3.py	问题三厚度计算代码
灵敏度分析.py	灵敏度分析计算代码
AI 工具使用详情.docx	AI 工具使用情况

附录 B　代码

拟合函数.py

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import least_squares
from scipy.optimize import fsolve

# 设置中文显示
plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False  # 解决负号显示问题

class FourierFitter:
    def __init__(self, n_terms=3):
        """
        初始化傅里叶拟合器
        n_terms: 傅里叶级数的项数，每一项包含一个正弦和一个余弦分量
        """
        self.n_terms = n_terms
        self.params = None
        self.period = None
```

```
        self.x_data = None
        self.y_data = None

    def _fourier_function(self, x, params, period):
        """
        傅里叶级数函数:
        y = a0 + Σ(an*cos(2πn x/period) + bn*sin(2πn x/period))
        其中n从1到n_terms
        """
        a0 = params[0]  # 直流分量
        result = a0

        for i in range(self.n_terms):
            an = params[1 + 2*i]    # 余弦项系数
            bn = params[2 + 2*i]    # 正弦项系数
            n = i + 1
            result += an * np.cos(2 * np.pi * n * x / period) + bn * np.sin(2 * np.pi * n * x / period)

        return result

    def _fourier_derivative(self, x, params, period):
        """
        傅里叶级数的一阶导数，用于寻找极值点
        y' = Σ(-an*(2πn/period)*sin(2πn x/period) + bn*(2πn/period)*cos(2πn x/period))
        """
        result = 0.0

        for i in range(self.n_terms):
            an = params[1 + 2*i]    # 余弦项系数
            bn = params[2 + 2*i]    # 正弦项系数
            n = i + 1
            omega = 2 * np.pi * n / period
```

```
            result += -an * omega * np.sin(omega * x) + bn * omega * np.cos(omega * x)

        return result

    def _residuals(self, params, x, y, period):
        """残差函数，用于最小二乘拟合"""
        return self._fourier_function(x, params, period) - y

    def fit(self, x_data, y_data):
        """拟合数据"""
        # 保存数据用于后续计算极值点
        self.x_data = x_data
        self.y_data = y_data

        # 估计周期（使用x数据范围作为初始估计）
        self.period = np.max(x_data) - np.min(x_data)

        # 初始参数猜测（全为1）
        # 参数包括: a0, a1, b1, a2, b2, ..., an, bn
        initial_params = np.ones(1 + 2 * self.n_terms)
        # 直流分量初始值设为y的平均值
        initial_params[0] = np.mean(y_data)

        # 使用最小二乘法拟合
        result = least_squares(
            self._residuals,
            initial_params,
            args=(x_data, y_data, self.period),
            method='lm'
        )

        self.params = result.x
        return self

```

```python
    def predict(self, x):
        """预测新数据点"""
        if self.params is None:
            raise ValueError("模型尚未拟合，请先调用fit方法")
        return self._fourier_function(x, self.params, self.period)

    def find_extrema(self, num_points=100):
        """
        寻找拟合函数在数据范围内的极值点（极大值和极小值）
        num_points: 用于寻找初始猜测点的数量
        """
        if self.params is None or self.x_data is None:
            raise ValueError("模型尚未拟合，请先调用fit方法")

        # 确定数据范围
        x_min, x_max = np.min(self.x_data), np.max(self.x_data)

        # 在数据范围内均匀取点，作为极值点的初始猜测
        x_guess = np.linspace(x_min, x_max, num_points)

        # 计算这些点的导数值
        derivatives = self._fourier_derivative(x_guess, self.params, self.period)

        # 寻找导数符号变化的区间（可能存在极值点）
        sign_changes = np.where(np.diff(np.sign(derivatives)))[0]

        # 求解每个区间内的极值点
        extrema_x = []
        for i in sign_changes:
            # 在符号变化的区间内寻找导数为零的点
            x0 = (x_guess[i] + x_guess[i+1]) / 2  # 区间中点作
```

```
为初始猜测
            root = fsolve(
                lambda x: self._fourier_derivative(x, self.params, self.period),
                x0
            )[0]

            # 确保解在数据范围内
            if x_min <= root <= x_max:
                extrema_x.append(root)

        # 去除非常接近的点（可能是数值误差导致的重复解）
        if extrema_x:
            extrema_x = np.unique(np.round(extrema_x, decimals=6))

        # 计算极值点对应的y值
        extrema_y = self.predict(extrema_x)

        # 区分极大值和极小值（通过二阶导数判断）
        # 计算二阶导数（一阶导数的导数）
        second_derivatives = []
        for x in extrema_x:
            # 使用数值方法计算二阶导数
            h = 1e-6
            dy1 = self._fourier_derivative(x + h, self.params, self.period)
            dy0 = self._fourier_derivative(x - h, self.params, self.period)
            second_deriv = (dy1 - dy0) / (2 * h)
            second_derivatives.append(second_deriv)

        # 分类：二阶导数为负是极大值，为正是极小值
        maxima = [(x, y) for x, y, d2 in zip(extrema_x, extrema_y, second_derivatives) if d2 < 0]
```

```python
        minima = [(x, y) for x, y, d2 in zip(extrema_x, extrema_y, second_derivatives) if d2 > 0]

        return {
            'all': list(zip(extrema_x, extrema_y)),
            'maxima': maxima,
            'minima': minima
        }

    def get_equation(self):
        """获取傅里叶级数方程字符串"""
        if self.params is None:
            raise ValueError("模型尚未拟合，请先调用fit方法")

        equation = f"y = {self.params[0]:.4f}"  # a0项

        for i in range(self.n_terms):
            an = self.params[1 + 2*i]
            bn = self.params[2 + 2*i]
            n = i + 1

            # 只添加系数绝对值大于1e-6的项，避免过多微小项
            if abs(an) > 1e-6:
                equation += f" + {an:.4f}·cos(2π·{n}·x/{self.period:.4f})"
            if abs(bn) > 1e-6:
                equation += f" + {bn:.4f}·sin(2π·{n}·x/{self.period:.4f})"

        return equation

    def plot(self, title="傅里叶级数拟合"):
        """绘制原始数据、拟合曲线及极值点"""
        if self.params is None:
            raise ValueError("模型尚未拟合，请先调用fit方法")
```

```python

        # 生成拟合曲线上的点
        x_fit = np.linspace(min(self.x_data), max(self.x_data), 1000)
        y_fit = self.predict(x_fit)

        # 寻找极值点
        extrema = self.find_extrema()

        # 绘制图形
        plt.figure(figsize=(12, 6))
        plt.scatter(self.x_data, self.y_data, s=5, label='原始数据', alpha=0.5)
        plt.plot(x_fit, y_fit, 'r-', linewidth=2, label='傅里叶拟合曲线')

        # 绘制极值点
        if extrema['maxima']:
            max_x, max_y = zip(*extrema['maxima'])
            plt.scatter(max_x, max_y, color='green', s=50, marker='^', label='极大值点')

        if extrema['minima']:
            min_x, min_y = zip(*extrema['minima'])
            plt.scatter(min_x, min_y, color='red', s=50, marker='v', label='极小值点')

        plt.xlabel('X轴数据')
        plt.ylabel('Y轴数据')
        plt.title(title)
        plt.legend()
        plt.grid(True)

        # 显示拟合方程（简化版，避免太长）
        short_eq = f"y = a0 + Σ(an · cos(2πnx/T) + bn · sin(2πnx/T
```

```
))"
        plt.text(0.05, 0.95, short_eq, transform=plt.gca().transAxes,
                 verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

        plt.tight_layout()
        plt.show()

        # 打印完整方程
        print("\n傅里叶级数拟合方程:")
        print(self.get_equation())
        print(f"\n周期 T = {self.period:.6f}")

        # 打印系数
        print("\n拟合系数:")
        print(f"a0 (直流分量): {self.params[0]:.6f}")
        for i in range(self.n_terms):
            an = self.params[1 + 2*i]
            bn = self.params[2 + 2*i]
            n = i + 1
            print(f"n={n}: an={an:.6f}, bn={bn:.6f}")

        # 打印极值点信息
        print("\n拟合函数在数据范围内的极值点:")
        print(f"极大值点数量: {len(extrema['maxima'])}")
        for i, (x, y) in enumerate(extrema['maxima'], 1):
            print(f"  极大值点 {i}: x={x:.6f}, y={y:.6f}")

        print(f"\n极小值点数量: {len(extrema['minima'])}")
        for i, (x, y) in enumerate(extrema['minima'], 1):
            print(f"  极小值点 {i}: x={x:.6f}, y={y:.6f}")
# ################################################################
```

```python
# ##############################################################
# 读取Excel数据 - 第2700行到第5395行
def read_excel_data(filename):
    try:
        # 第2700行到第5395行
        skip_rows = 0  # 跳过前2699行
        num_rows = 7000  # 读取2696行数据

        df = pd.read_excel(
            filename,
            usecols=[0, 1],  # 选择对应列
            skiprows=skip_rows,
            nrows=num_rows
        )

        x_data = df.iloc[:, 0].values  # 第一列数据作为x
        y_data = df.iloc[:, 1].values  # 第二列数据作为y

        print(f"成功读取数据: {len(x_data)}个数据点 (第2700行到第5395行)")
        return x_data, y_data

    except FileNotFoundError:
        print(f"错误: 找不到文件 '{filename}'")
        return None, None
    except Exception as e:
        print(f"读取文件时发生错误: {str(e)}")
        return None, None

# 主函数
def main():
    filename = "附件4.xlsx"
    print(f"正在读取文件: {filename}")
```

```python

    x_data, y_data = read_excel_data(filename)

    if x_data is not None and y_data is not None:
        # 可以通过调整n_terms来改变拟合精度
        n_terms = 40  # 傅里叶级数的项数
        print(f"使用 {n_terms} 项傅里叶级数进行拟合...")

        fitter = FourierFitter(n_terms=n_terms)
        fitter.fit(x_data, y_data)
        fitter.plot(title="傅里叶级数拟合 (第2700行到第5395行数据)")

if __name__ == "__main__":
    main()
```

折射率的计算.py

```python
import math
import pandas as pd
from scipy.optimize import fsolve

def equation(n1, R, theta0):
    """定义需要求解的方程，返回方程值"""
    # 计算sin(theta0)
    sin_theta0 = math.sin(theta0)

    # 避免出现复数，计算根号内的值
    sqrt_arg = 1 - (sin_theta0**2) / (n1**2)

    # 如果根号内为负数，返回一个大值表示不满足条件
    if sqrt_arg < 0:
        return 1e10

    sqrt_val = math.sqrt(sqrt_arg)
    cos_theta0 = math.cos(theta0)
```

```python

    # 计算第一个分数项
    term1_numerator = cos_theta0 - n1 * sqrt_val
    term1_denominator = cos_theta0 + n1 * sqrt_val
    term1 = (term1_numerator / term1_denominator)** 2

    # 计算第二个分数项
    term2_numerator = n1 * cos_theta0 - sqrt_val
    term2_denominator = n1 * cos_theta0 + sqrt_val
    term2 = (term2_numerator / term2_denominator)**2

    # 计算R值
    calculated_R = 0.5 * (term1 + term2)

    # 返回方程值，目标是使其为0
    return calculated_R - R

def calculate_optical_parameters(R, theta0_deg, m, nu_m, initial_guess=1.5):
    """
    计算光学参数

    参数:
    R: 已知的R值
    theta0_deg: 已知的theta0值，单位为度
    m: 已知量m
    nu_m: 已知量nu_m
    initial_guess: n1的初始猜测值

    返回:
    包含所有计算结果的字典
    """
    # 将角度转换为弧度
    theta0_rad = math.radians(theta0_deg)

```

```python
    # 计算sin(theta0)
    sin_theta0 = math.sin(theta0_rad)

    # 使用fsolve求解方程
    n1_solution, = fsolve(equation, initial_guess, args=(R, theta0_rad))

    # 计算sin(theta1)和cos(theta1)
    sin_theta1 = sin_theta0 / n1_solution
    cos_theta1 = math.sqrt(1 - sin_theta1**2)  # 由三角恒等式sin²+cos²=1

    # 计算d值，公式：d = (m - 1/2) / (2 * n1 * cos(theta1) * nu_m)
    d_value = 1 / (2 * n1_solution * cos_theta1 * nu_m)

    # 验证结果
    residual = equation(n1_solution, R, theta0_rad)

    # 将结果整理为字典返回
    return {
        'R': R,
        'theta0_deg': theta0_deg,
        'm': m,
        'nu_m': nu_m,
        'n1': n1_solution,
        'sin_theta0': sin_theta0,
        'sin_theta1': sin_theta1,
        'cos_theta1': cos_theta1,
        'd': d_value,
        'residual': residual
    }

def get_available_sheets(file_path):
    """获取Excel文件中所有可用的sheet名称"""
```

```
    try:
        excel_file = pd.ExcelFile(file_path)
        return excel_file.sheet_names
    except Exception as e:
        print(f"获取sheet列表时出错: {str(e)}")
        return []

def process_excel_file(file_path='result.xlsx', sheet_name=0):
    """
    处理Excel文件，读取指定sheet的数据，计算R值，然后计算结果

    参数:
    file_path: Excel文件路径
    sheet_name: 要使用的sheet名称或索引（默认为0，即第一个sheet）

    返回:
    包含所有计算结果的DataFrame
    """
    try:
        # 读取Excel文件中指定的sheet，第1-4列分别为用于计算R的基础值、theta0、m、nu_m
        # 不使用表头，直接按列索引读取
        df = pd.read_excel(file_path, sheet_name=sheet_name, header=None, usecols=[0, 1, 2, 3])
        df.columns = ['R_base', 'theta0', 'm', 'nu_m']

        # 计算R值：第i个R值为第一列第i行值与第i+1行值的均值
        # 最后一行没有i+1行，所以不处理
        results = []

        # 逐行处理数据（除了最后一行）
        for i in range(len(df) - 1):
            try:
                # 计算R值：当前行与下一行的平均值
```

```
                R = (df.iloc[i]['R_base'] + df.iloc[i+1]['R_base']) / 2

                # 提取其他数据
                theta0 = df.iloc[i]['theta0']
                m = df.iloc[i]['m']
                nu_m = df.iloc[i]['nu_m']

                # 计算光学参数
                result = calculate_optical_parameters(R, theta0, m, nu_m)
                # 记录当前行索引，方便核对
                result['row_index'] = i + 1  # 行号从1开始
                results.append(result)

                print(f"已处理第{i+1}行数据 (R = 第{i+1}行与第{i+2}行的平均值)")
            except Exception as e:
                print(f"处理第{i+1}行数据时出错: {str(e)}")

        # 将结果转换为DataFrame并返回
        results_df = pd.DataFrame(results)
        # 调整列顺序，方便查看
        if not results_df.empty:
            results_df = results_df[['row_index', 'R', 'theta0_deg', 'm', 'nu_m',
                                     'n1', 'sin_theta0', 'sin_theta1', 'cos_theta1',
                                     'd', 'residual']]
        return results_df

    except Exception as e:
        print(f"处理Excel文件时出错: {str(e)}")
        return None

```

```python
if __name__ == "__main__":
    file_path = 'result.xlsx'

    # 获取并显示所有可用的sheet
    sheets = get_available_sheets(file_path)
    if sheets:
        print(f"文件 '{file_path}' 中包含以下sheet:")
        for i, sheet in enumerate(sheets):
            print(f"  {i}: {sheet}")

        # 让用户选择sheet
        while True:
            try:
                choice = input("\n请输入要使用的sheet索引或名称: ")
                # 尝试将输入转换为索引
                if choice.isdigit():
                    sheet_choice = int(choice)
                    if 0 <= sheet_choice < len(sheets):
                        break
                    else:
                        print(f"无效的索引，请输入0到{len(sheets)-1}之间的数字")
                else:
                    # 检查输入是否为有效的sheet名称
                    if choice in sheets:
                        sheet_choice = choice
                        break
                    else:
                        print(f"无效的名称，请从上述列表中选择一个sheet名称")
            except Exception as e:
                print(f"输入错误: {str(e)}，请重试")

        # 处理选定的sheet并获取结果
```

```
        print(f"\n正在处理sheet: {sheet_choice}")
        results_df = process_excel_file(file_path, sheet_choice)

        if results_df is not None and not results_df.empty:
            # 显示结果
            print("\n计算结果:")
            print(results_df)

            # 将结果保存到新的Excel文件
            output_file = 'calculation_results.xlsx'
            results_df.to_excel(output_file, index=False)
            print(f"\n结果已保存到 {output_file}")
        elif results_df is not None:
            print("\n没有可处理的数据行，请检查Excel文件内容")
    else:
        print(f"无法读取文件 '{file_path}' 中的sheet，请检查文件是否存在且格式正确")
```

折射率的计算 _ 本质法.py

```
import math
import cmath
from typing import Union
import pandas as pd  # 用于数据处理和Excel写入

def calculate_6H_SiC_refractive_index(
    wavenumber_cm: Union[float, int],
    N: float = 2e15
) -> complex:
    """
    基于文献《Thickness Determination...》计算6H-SiC的复数折射率
    改进说明:
    1. 支持float类型的波数输入
    2. 增强参数校验和异常处理
```

```
    3. 优化数值计算稳定性

    参数:
        wavenumber_cm: 波数, 支持小数 (单位cm□¹, 建议200-600 cm□¹范围)
        N: 载流子浓度 (单位cm□³, 默认2e15)
    返回:
        复数折射率 n = n' + i*k (垂直与平行分量的平均值)
    异常:
        ValueError: 当波数超出建议范围时警告
    """
    # 参数校验 (非强制限制, 仅提示)
    if not (200 <= wavenumber_cm <= 600):
        print(f"警告: 波数{wavenumber_cm}cm□¹超出建议范围(200-600 cm□¹), 计算结果可能不准确")

    # 物理常数 (使用更高精度的CODATA推荐值)
    e = 1.602176634e-19        # 电子电荷(C) [2018 CODATA]
    m0 = 9.1093837015e-31      # 电子质量(kg) [2018 CODATA]
    c = 2.99792458e10          # 光速(cm/s) [精确值]
    eps0 = 8.8541878128e-12    # 真空介电常数(F/m) [2018 CODATA]

    # 6H-SiC参数 (来自文献表1)
    # 垂直分量参数 (⊥)
    #eps_inf_perp = 6.52
    #omega_TO_perp = 793.9     # cm□¹
    #omega_LO_perp = 970.1     # cm□¹
    #gamma_perp = 4.763        # cm□¹
    #m_star_perp = 0.42 * m0

    # 平行分量参数 (∥)
    #eps_inf_paral = 6.70
    #omega_TO1_paral = 785.9   # cm□¹
    #omega_LO1_paral = 959.7   # cm□¹
```

```
    #gamma1_paral = 5.501      # cm□¹
    #omega_TO2_paral = 882.9   # cm□¹
    #omega_LO2_paral = 887.9   # cm□¹
    #gamma2_paral = 4.856      # cm□¹
    #m_star_paral = 1.83 * m0

    # 自由载流子参数
    #gamma_drude = 757.0       # cm□¹

    # 4H-SiC参数（来自文献表1）
    # 垂直分量参数 (⊥)
    eps_inf_perp = 6.52
    omega_TO_perp = 793.9     # cm□¹
    omega_LO_perp = 970.1     # cm□¹
    gamma_perp = 4.76        # cm□¹
    m_star_perp = 0.4 * m0

    # 平行分量参数 (∥)
    eps_inf_paral = 6.70
    omega_TO1_paral = 785.9   # cm□¹
    omega_LO1_paral = 959.7   # cm□¹
    gamma1_paral = 5.501      # cm□¹
    omega_TO2_paral = 882.9   # cm□¹
    omega_LO2_paral = 887.9   # cm□¹
    gamma2_paral = 4.856      # cm□¹
    m_star_paral = 0.31 * m0

    # 自由载流子参数
    gamma_drude = 757.0       # cm□¹

    def calculate_complex_epsilon(
        wavenumber: float,
        eps_inf: float,
        TO_list: list,
        LO_list: list,
```

```
        Gamma_list: list,
        m_star: float
    ) -> complex:
        """
        改进的复数介电函数计算（支持小数波数）
        关键改进：
        1. 使用cmath处理复数运算
        2. 增加数值稳定性检查
        3. 优化频率单位转换
        """
        omega = 2 * math.pi * c * wavenumber  # 角频率(rad/s)

        # 单位转换 (cm□¹ → rad/s)
        TO_rad = [2 * math.pi * c * to for to in TO_list]
        LO_rad = [2 * math.pi * c * lo for lo in LO_list]
        Gamma_rad = [2 * math.pi * c * gamma for gamma in Gamma_list]
        gamma_drude_rad = 2 * math.pi * c * gamma_drude

        # 计算Lorentz项（改进数值稳定性）
        lorentz = 0.0
        for to, lo, gamma in zip(TO_rad, LO_rad, Gamma_rad):
            denominator = to**2 - omega**2 - 1j*gamma*omega
            # 避免除以零（当ω接近ω_TO时添加微小偏移）
            if abs(denominator) < 1e-10:
                denominator += 1j*1e-10
            lorentz += (lo**2 - to**2) / denominator

        # 计算Drude项（改进单位一致性）
        omega_p_sq = (4 * math.pi * N * e**2) / (m_star * eps0 * 100**3)  # cm□³→m□3
        drude_denominator = omega * (omega + 1j*gamma_drude_rad)
        drude = -omega_p_sq / drude_denominator

```

```
        return eps_inf * (1 + lorentz + drude)

    # 计算垂直分量（增加计算过程保护）
    try:
        epsilon_perp = calculate_complex_epsilon(
            wavenumber=float(wavenumber_cm),  # 显式转换为float
            eps_inf=eps_inf_perp,
            TO_list=[omega_TO_perp],
            LO_list=[omega_LO_perp],
            Gamma_list=[gamma_perp],
            m_star=m_star_perp
        )
        n_perp = cmath.sqrt(epsilon_perp)
    except Exception as e:
        raise ValueError(f"垂直分量计算错误: {str(e)}")

    # 计算平行分量（增加计算过程保护）
    try:
        epsilon_paral = calculate_complex_epsilon(
            wavenumber=float(wavenumber_cm),
            eps_inf=eps_inf_paral,
            TO_list=[omega_TO1_paral, omega_TO2_paral],
            LO_list=[omega_LO1_paral, omega_LO2_paral],
            Gamma_list=[gamma1_paral, gamma2_paral],
            m_star=m_star_paral
        )
        n_paral = cmath.sqrt(epsilon_paral)
    except Exception as e:
        raise ValueError(f"平行分量计算错误: {str(e)}")

    # 返回平均值（实部和虚部分别平均）
    return (n_perp + n_paral) / 2

# 测试代码（包含小数波数案例并保存到Excel）
```

```python
if __name__ == "__main__":
    test_wavenumbers = [450.826657
,752.409146
,1123.260186
,1506.605984
,1951.195069
,2338.060197
,567.618933
,942.37898
,1316.742408
,1710.050347
,2148.870447
,2527.039045
]
    results = []  # 用于存储计算结果

    print("6H-SiC复数折射率计算（支持小数波数）")
    print("波数(cm□¹) | 折射率实部n' | 消光系数k")
    print("-" * 45)

    for wn in test_wavenumbers:
        try:
            n_complex = calculate_6H_SiC_refractive_index(wn)
            real_part = n_complex.real
            imag_part = n_complex.imag
            print(f"{wn:10.2f} | {real_part:.6f}  | {imag_part:.6f}")
            # 将结果添加到列表
            results.append({
                '波数(cm□¹)': wn,
                '折射率实部n\'': real_part,
                '消光系数k': imag_part
            })
        except ValueError as e:
            print(f"{wn:10.2f} | 计算失败: {str(e)}")
```

```
            # 记录失败的计算
            results.append({
                '波数(cm□¹)': wn,
                '折射率实部n\'': '计算失败',
                '消光系数k': str(e)
            })

    # 创建DataFrame并保存到Excel
    df = pd.DataFrame(results)
    df.to_excel('result_f.xlsx', index=False, engine='openpyxl')
    print(f"\n计算结果已保存到 result_f.xlsx")
```

Q3 硅的折射率计算.py

```
import numpy as np
import pandas as pd

def sellmeier_si(wavelength_um):
    """根据Sellmeier公式计算硅的折射率"""
    term1 = 11.67316
    term2 = 1 / (wavelength_um ** 2)
    term3 = 0.004482633 / (wavelength_um **2 - 1.108205** 2)
    return np.sqrt(term1 + term2 + term3)

# 读取Excel文件（假设波数数据在"波数(cm□¹)"列，可根据实际列名修改）
df = pd.read_excel("boshu.xlsx")

# 计算折射率（波数转波长：λ(μm) = 10^4 / 波数(cm□¹)）
df["折射率n"] = df["波数(cm□¹)"].apply(lambda sigma: sellmeier_si(10**4 / sigma))

# 保存结果回原Excel文件
df.to_excel("boshu.xlsx", index=False)

```

```python
print("计算完成，结果已保存到boshu.xlsx")
```

Q3 碳化硅折射率的计算.py

```python
import pandas as pd
import math
import os

def calculate_average_refractive_index(wavenumber):
    """计算给定波数下s和p偏振折射率的算术平均值"""
    # 波数转波长（单位：μm）
    lambda_μm = 10**4 / wavenumber

    # s和p偏振的参数
    s_A, s_B = 5.67, 0.034
    p_A, p_B = 5.59, 0.029

    # 计算两种偏振的折射率
    s_n = math.sqrt(1 + (s_A * lambda_μm**2) / (lambda_μm**2 - s_B))
    p_n = math.sqrt(1 + (p_A * lambda_μm**2) / (lambda_μm**2 - p_B))

    # 返回算术平均值
    return (s_n + p_n) / 2

def process_excel_file(filename):
    """读取Excel中的波数，计算折射率并写回"""
    try:
        # 检查文件是否存在
        if not os.path.exists(filename):
            print(f"错误：文件 '{filename}' 不存在")
            return

        # 读取Excel文件
        df = pd.read_excel(filename)
```

```python

        # 检查是否存在波数列（假设列名为"波数"或"波数(cm□¹)"）
        wave_number_columns = [col for col in df.columns if '波数' in str(col)]
        if not wave_number_columns:
            print("错误：Excel文件中未找到包含'波数'的列")
            return

        # 使用找到的第一个波数列
        wave_number_col = wave_number_columns[0]
        print(f"使用列 '{wave_number_col}' 中的波数数据进行计算")

        # 计算折射率并添加到DataFrame
        df['平均折射率'] = df[wave_number_col].apply(calculate_average_refractive_index)

        # 将结果写回Excel文件
        df.to_excel(filename, index=False)
        print(f"计算完成，结果已保存到 '{filename}' 中的'平均折射率'列")

        # 只输出折射率结果
        print("\n计算得到的平均折射率：")
        print(df[['平均折射率']].to_string(index=False))

    except Exception as e:
        print(f"处理过程中发生错误：{str(e)}")

# 主程序
if __name__ == "__main__":
    excel_filename = "boshu.xlsx"
    process_excel_file(excel_filename)
```

Q3.py

```python
import pandas as pd
import numpy as np
from scipy.optimize import minimize, differential_evolution
import math
import matplotlib.pyplot as plt

# 全局设置
plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False
REFLECTIVITY_MIN = 0.0
REFLECTIVITY_MAX = 1.0


def preprocess_data(df):
    """增强版数据预处理：剔除异常值+平滑噪声"""
    wavenumbers = df.iloc[:, 0].values
    angles = df.iloc[:, 1].values
    substrate_n = df.iloc[:, 2].values  # 允许为复数（若有吸收）
    reflectivity = df.iloc[:, 3].values

    # 1. 转换衬底折射率为复数（若输入为实数则虚部为0）
    substrate_n = np.array(substrate_n, dtype=np.complex128)

    # 2. 剔除反射率异常值
    mask = (reflectivity >= REFLECTIVITY_MIN) & (reflectivity <= REFLECTIVITY_MAX)
    if len(reflectivity[mask]) > 5:
        # 用中位数绝对偏差法剔除离群值
        med = np.median(reflectivity[mask])
        mad = np.median(np.abs(reflectivity[mask] - med))
        mask &= (np.abs(reflectivity - med) <= 3 * mad)

    # 3. 按波数排序
    sorted_idx = np.argsort(wavenumbers[mask])
```

```
    return (wavenumbers[mask][sorted_idx],
            angles[mask][sorted_idx],
            substrate_n[mask][sorted_idx],
            reflectivity[mask][sorted_idx])


def calculate_reflectivity(params, wavenumber, angle, substrate_n):
    """
    修正版反射率计算:
    1. 外延层折射率为复数n1 = n_real - i*k (含吸收)
    2. 严格处理波数与波长的单位转换
    """
    n_real, k, d_cm = params  # d_cm: 厚度(cm)
    n1 = complex(n_real, -k)  # 复数折射率 (吸收项为正)
    n0 = 1.0  # 空气折射率 (实数)
    theta0 = math.radians(angle)

    # 计算折射角 (复数折射率的折射定律)
    sin_theta1 = math.sin(theta0) / n1.real  # 用实部计算折射角 (近似)
    sin_theta1 = np.clip(sin_theta1, -1.0 + 1e-10, 1.0 - 1e-10)
    theta1 = math.asin(sin_theta1)
    cos_theta1 = math.cos(theta1)

    # 衬底折射角 (若衬底有吸收, 同样用实部)
    sin_theta2 = math.sin(theta0) / substrate_n.real
    sin_theta2 = np.clip(sin_theta2, -1.0 + 1e-10, 1.0 - 1e-10)
    theta2 = math.asin(sin_theta2)
    cos_theta2 = math.cos(theta2)

    # 菲涅尔系数 (自然光: s和p平均)
    # s偏振
```

```
    r1_s = (n0*math.cos(theta0) - n1*cos_theta1) / (n0*math.cos(theta0) + n1*cos_theta1)
    r2_s = (n1*cos_theta1 - substrate_n*cos_theta2) / (n1*cos_theta1 + substrate_n*cos_theta2)
    # p偏振
    r1_p = (n1*math.cos(theta0) - n0*cos_theta1) / (n1*math.cos(theta0) + n0*cos_theta1)
    r2_p = (substrate_n*cos_theta1 - n1*cos_theta2) / (substrate_n*cos_theta1 + n1*cos_theta2)
    # 自然光平均（取模值，因反射系数为复数）
    r1 = (np.abs(r1_s) + np.abs(r1_p)) / 2
    r2 = (np.abs(r2_s) + np.abs(r2_p)) / 2

    # 相位差计算（关键修正：波数单位转换）
    wavelength_cm = 1 / wavenumber  # 波长(cm) = 1/波数(cm□¹)
    k0 = 2 * math.pi / wavelength_cm  # 真空中的波数(rad/cm)
    delta = k0 * n1.real * d_cm * cos_theta1  # 光程差对应的相位（用实部计算）

    # Δφ判断（用折射率实部比较）
    delta_phi = math.pi if n1.real > substrate_n.real else 0.0

    # 反射率公式（含复数模值处理）
    cos_term = math.cos(2 * delta - delta_phi)
    numerator = r1**2 + r2**2 + 2 * r1 * r2 * cos_term
    denominator = 1 + (r1 * r2)** 2 + 2 * r1 * r2 * cos_term
    return np.clip(numerator / denominator, REFLECTIVITY_MIN, REFLECTIVITY_MAX)


def residuals(params, wavenumbers, angles, substrate_n, reflectivity):
    """带相位约束的残差函数"""
    theoretical = np.array([
        calculate_reflectivity(params, wn, ang, sub_n)
```

```
        for wn, ang, sub_n in zip(wavenumbers, angles, substrate_n)
    ])
    # 动态权重：残差越大权重越高（加速收敛）
    weights = 1.0 + 10 * np.abs(reflectivity - 0.5)  # 对中间反射率点提高权重
    return np.sum(weights * (theoretical - reflectivity)**2)


def optimize_parameters(wavenumbers, angles, substrate_n, reflectivity):
    """
    周期约束优化：
    1. 先估算厚度范围（基于最小波数对应的半波长）
    2. 结合全局+局部优化
    """
    # 估算厚度初始范围（基于最长波长的1/4 ~ 5倍）
    min_wavenumber = np.min(wavenumbers)
    max_wavelength_cm = 1 / min_wavenumber  # 最长波长(cm)
    n_guess = 3.0  # 折射率初猜
    d_min = 1e-4#max_wavelength_cm / (4 * n_guess) * 0.5  # 1/8波长
    d_max = 1e-3#max_wavelength_cm / (4 * n_guess) * 10   # 2.5波长

    # 参数边界（n_real: 1.8-4.5, k: 0-0.1（弱吸收）, d: d_min-d_max）
    bounds = [
        (1.8, 4.5),
        (0.0, 0.1),
        (d_min, d_max)
    ]

    # 1. 全局优化（寻找大致范围）
    global_res = differential_evolution(
```

```
        residuals, bounds,
        args=(wavenumbers, angles, substrate_n, reflectivity),
        maxiter=500, popsize=30, mutation=(0.5, 1.5)
    )

    # 2. 局部优化（精细调整）
    local_res = minimize(
        residuals, global_res.x,
        args=(wavenumbers, angles, substrate_n, reflectivity),
        method='L-BFGS-B', bounds=bounds,
        options={'maxiter': 10000, 'gtol': 1e-10}
    )

    return local_res.x if local_res.success else global_res.x


def plot_results(wavenumbers, reflectivity, fitted, params):
    n_real, k, d_cm = params
    d_microns = d_cm * 1e4  # cm→μm
    rmse = np.sqrt(np.mean((reflectivity - fitted)**2))
    r2 = 1 - np.sum((reflectivity - fitted)** 2) / np.sum((reflectivity - np.mean(reflectivity))**2)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 9), sharex=True)
    fig.suptitle(
        f'拟合结果（n={n_real:.4f}, k={k:.6f}, d={d_microns:.4f}μm | RMSE={rmse:.6f}）',
        fontsize=14
    )

    # 实验vs拟合
    ax1.scatter(wavenumbers, reflectivity, c='blue', s=30, alpha=0.7, label='实验值')
    ax1.plot(wavenumbers, fitted, 'r-', linewidth=2, label='拟
```

```
    合值')
    ax1.set_ylabel('反射率')
    ax1.set_title(f'R²={r2:.4f}')
    ax1.legend()
    ax1.grid(alpha=0.3)
    ax1.set_ylim(-0.05, 1.05)

    # 残差
    residuals = reflectivity - fitted
    ax2.scatter(wavenumbers, residuals, c='green', s=30, alpha=0.7, label='残差')
    ax2.axhline(y=0, c='k', ls='--', alpha=0.3)
    ax2.set_xlabel('波数 (cm⁻¹)')
    ax2.set_ylabel('残差')
    ax2.legend()
    ax2.grid(alpha=0.3)

    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.savefig('高精度拟合结果.png', dpi=300)
    plt.show()


def main():
    try:
        df = pd.read_excel("input3.xlsx")
        wavenumbers, angles, substrate_n, reflectivity = preprocess_data(df)
        if len(wavenumbers) < 8:
            print("警告: 数据点少于8个, 拟合可靠性低")
            return
    except Exception as e:
        print(f"数据处理失败: {e}")
        return

    try:
```

```
        best_params = optimize_parameters(wavenumbers, angles, substrate_n, reflectivity)
        n_real, k, d_cm = best_params
        d_microns = d_cm * 1e4
    except Exception as e:
        print(f"优化失败: {e}")
        return

    # 计算拟合反射率
    fitted = np.array([
        calculate_reflectivity(best_params, wn, ang, sub_n)
        for wn, ang, sub_n in zip(wavenumbers, angles, substrate_n)
    ])

    # 输出结果
    print("\n拟合参数: ")
    print(f"外延层折射率（实部）: {n_real:.4f}")
    print(f"吸收系数（虚部）: {k:.6f}")
    print(f"厚度: {d_microns:.4f} μm")
    print(f"RMSE: {np.sqrt(np.mean((reflectivity - fitted)**2)):.6f}")

    # 绘图与保存
    plot_results(wavenumbers, reflectivity, fitted, best_params)
    output_df = pd.DataFrame({
        '波数(cm^-1)': wavenumbers,
        '入射角(度)': angles,
        '衬底折射率': [f"{n.real:.4f}" for n in substrate_n],
        '实验反射率': reflectivity,
        '拟合反射率': fitted,
        '残差': reflectivity - fitted
    })
    params_df = pd.DataFrame({
```

```
        '波数(cm^-1)': ['折射率实部', '吸收系数(k)', '厚度(μm)', 'RMSE'],
        '入射角(度)': [n_real, k, d_microns, np.sqrt(np.mean((reflectivity - fitted)**2))],
        '衬底折射率': ['', '', '', ''],
        '实验反射率': ['', '', '', ''],
        '拟合反射率': ['', '', '', ''],
        '残差': ['', '', '', '']
    })
    output_df = pd.concat([output_df, params_df], ignore_index=True)
    output_df.to_excel("高精度拟合结果.xlsx", index=False)


if __name__ == "__main__":
    main()
```

灵敏度分析.py

```
import math

def calculate_thickness(n2, cos_theta2, delta_nu):

    d_cm = 1 / (2 * n2 * cos_theta2 * delta_nu)
    d_um = d_cm * 1e4  # cm转μm（1cm=1e4μm）
    return d_um

# 文档中问题二的基准参数（用于灵敏度分析）
base_params = {
    "n2": 2.48,
    "cos_theta2": 0.9975,
    "delta_nu": 251.0,
    "d_base": 8.03

disturbance_schemes = {
    "n2": [(0.001, "n2_+0.1%"), (-0.003, "n2_-0.3%"), (0.005,
```

```
    "n2_+0.5%")],
     "cos_theta2": [(0.001, "cosθ2_+0.1%"), (-0.003, "cosθ2_-0.3%"), (0.005, "cosθ2_+0.5%")],
     "delta_nu": [(0.001, "Δν_+0.1%"), (-0.003, "Δν_-0.3%"), (0.005, "Δν_+0.5%")]
}

print("问题一灵敏度分析结果（与文档一致）\n")

print(f"{'参数':<12}{'扰动幅度':<10}{'扰动后值':<12}{'计算厚度(μm)':<15}{'厚度变化率':<12}")

print("-" * 60)  # 调整分隔线长度以匹配新表格

for param_name, schemes in disturbance_schemes.items():

    base_val = base_params[param_name]

    for disturb_ratio, disturb_label in schemes:
        # 计算扰动后参数值
        disturbed_val = base_val * (1 + disturb_ratio)
        # 构造当前参数组合（仅扰动目标参数，其余用基准值）
        current_n2 = disturbed_val if param_name == "n2" else base_params["n2"]
        current_cos = disturbed_val if param_name == "cos_theta2" else base_params["cos_theta2"]
        current_delta = disturbed_val if param_name == "delta_nu" else base_params["delta_nu"]
        # 计算扰动后厚度
        d_disturbed = calculate_thickness(current_n2, current_cos, current_delta)
        # 计算厚度变化率（保留3位小数，与文档精度一致）
        d_change_rate = ((d_disturbed - base_params["d_base"]) / base_params["d_base"]) * 100

```

```
        # 格式化输出（匹配文档数据精度）
        disturb_pct = f"{disturb_ratio*100:+.1f}%"
        disturbed_val_str = f"{disturbed_val:.6f}" if param_name == "cos_theta2" else f"{disturbed_val:.3f}"
        d_disturbed_str = f"{d_disturbed:.3f}"
        d_change_rate_str = f"{d_change_rate:+.3f}%"

        # 输出问题一数据
        print(f"{param_name:<12}{disturb_pct:<10}{disturbed_val_str:<12}{d_disturbed_str:<15}{d_change_rate_str:<12}")

# 输出问题三灵敏度表格（直接按图中数据编写）
print("\n问题三灵敏度分析结果\n")
print(f"{'参数':<12}{'基准值':<10}{'扰动幅度':<10}{'扰动后值':<12}{'计算厚度(μm)':<15}{'厚度变化率':<12}")
print("-" * 80)
# 定义问题三表格数据
problem3_data = [
    {"参数": "n□", "基准值": 2.58, "扰动幅度": "+0.1%", "扰动后值": 2.58258, "计算厚度(μm)": 5.1748, "厚度变化率": "-0.10%"},
    {"参数": "n□", "基准值": 2.58, "扰动幅度": "-0.3%", "扰动后值": 2.57226, "计算厚度(μm)": 5.1958, "厚度变化率": "+0.31%"},
    {"参数": "n□", "基准值": 2.58, "扰动幅度": "+0.5%", "扰动后值": 2.5929, "计算厚度(μm)": 5.1649, "厚度变化率": "-0.52%"},
    {"参数": "cosθ□", "基准值": 0.9986, "扰动幅度": "+0.1%", "扰动后值": 0.9995986, "计算厚度(μm)": 5.1748, "厚度变化率": "-0.10%"},
    {"参数": "cosθ□", "基准值": 0.9986, "扰动幅度": "-0.3%", "扰动后值": 0.9956042, "计算厚度(μm)": 5.1958, "厚度变化率": "+0.31%"},
    {"参数": "cosθ□", "基准值": 0.9986, "扰动幅度": "+0.5%", "
```

```
    扰动后值": 1.003593, "计算厚度(μm)": 5.1649, "厚度变化率": "-0.52%"},
     {"参数": "Δ□ (cm□¹)", "基准值": 195, "扰动幅度": "+0.1%", "扰动后值": 195.195, "计算厚度(μm)": 5.1748, "厚度变化率": "-0.10%"},
     {"参数": "Δ□ (cm□¹)", "基准值": 195, "扰动幅度": "-0.3%", "扰动后值": 194.415, "计算厚度(μm)": 5.1958, "厚度变化率": "+0.31%"},
     {"参数": "Δ□ (cm□¹)", "基准值": 195, "扰动幅度": "+0.5%", "扰动后值": 195.975, "计算厚度(μm)": 5.1649, "厚度变化率": "-0.52%"}
]
# 输出问题三表格数据
for data in problem3_data:
    print(f"{data['参数']:<12}{data['基准值']:<10}{data['扰动幅度']:<10}{data['扰动后值']:<12}{data['计算厚度(μm)']:<15}{data['厚度变化率']:<12}")
```